

PlanifPro

Application de gestion de Planning pédagogique

Documentation technique

I . User Stories

Must have : Indispensable :

User-01 :

En tant que professeur, je veux pouvoir créer un compte, me connecter et déconnecter de manière sécurisée, afin d'accéder à mon espace de gestion.

- Critère : inscription avec email/mot de passe, connexion et déconnexion fonctionnelle avec JWT.

User-02 :

En tant qu'élève ou parent, je veux pouvoir créer un compte, me connecter et me déconnecter, afin d'accéder à mon espace personnel.

- Critère : inscription avec email/mot de passe, connexion et déconnexion fonctionnelle avec JWT.

User-03 :

En tant que professeur, je veux voir un planning vide (type Google Calendar) avec les actions disponibles lors de ma première connexion, afin de comprendre rapidement comment configurer mon espace.

- Critère : calendrier FullCalendar affiché dès la première connexion, vide, avec des indications visuelles des actions disponibles (ex : "Créez votre première classe", "Ajoutez vos élèves") (écran d'onboarding affiché uniquement à la première connexion avec les options disponibles).

User-04 :

En tant qu'élève ou parent, je veux voir un planning vide (type Google Calendar) avec les actions disponibles lors de ma première connexion, afin de comprendre rapidement comment remplir mes informations.

- Critère : calendrier FullCalendar affiché dès la première connexion, vide, avec des indications visuelles des actions disponibles (ex : "Rejoignez une classe", "Soumettez vos vœux") (écran d'onboarding affiché uniquement à la première connexion avec les options disponibles).

User-05 :

En tant que professeur, je veux créer une ou plusieurs classes en définissant sa catégorie, sa période, ses jours et plages horaires ainsi que les contraintes de vœux, afin de préparer la génération du planning ou le placement des élèves manuellement dans mon planning. Un code unique est généré automatiquement à la création de la classe.

- Critère : création de classe avec catégorie, dates de début/fin, jours et plages horaires, nombre de vœux et jours minimum configurables. Selon la catégorie, le professeur pourra soit générer le planning via l'algorithme (vœux élèves), soit placer les créneaux manuellement (cours privés).
- Un code unique est généré automatiquement à la création de la classe.

User-06 :

En tant que professeur, je veux partager le code de ma classe (publique ou privée) ou envoyer une invitation par email directement depuis l'application, à mes élèves, afin qu'ils puissent me rejoindre.

- Critère : code unique généré automatiquement à la création de la classe, copiable et partageable par le professeur.
- Option d'envoi d'une invitation par email depuis l'application avec le code de classe inclus.

User-07 :

En tant qu'élève ou parent, je veux rejoindre la classe de mon professeur en entrant un code de classe, afin d'être rattaché automatiquement à la bonne classe.

- Critère : saisie du code de classe depuis le tableau de bord, rattachement automatique à la classe et au professeur.

User-08 :

En tant que professeur, je veux configurer la durée de cours d'un élève après qu'il ait rejoint ma classe, afin de personnaliser le planning selon son niveau.

- Critère : accès à la fiche de l'élève depuis la classe, durée de cours configurable (ex : 30 min, 45 min, 1h).

User-09 :

En tant que professeur, je veux lancer la collecte des vœux pour une classe, afin que mes élèves soient notifiés et puissent soumettre leurs disponibilités.

- Critère : bouton "Lancer la collecte des vœux", notification envoyée à tous les élèves de la classe.

User-10 :

En tant qu'élève ou parent, je veux renseigner mon planning personnel manuellement ou via import (Google Calendar).

- Critère : saisie manuelle de plages du planning personnel + option d'import du planning personnel (ex : Google Calendar).

User-11 :

En tant qu'élève ou parent, je veux soumettre mes vœux d'horaires (ex : 3 minimum sur 2 jours différents minimum), afin que mes disponibilités soient prises en compte dans la génération de planning.

- Critère : Formulaire de saisie de vœux, validation avec contrainte défini par le professeur.

User-12 :

En tant que professeur, je veux voir quels élèves ont soumis leurs vœux et relancer ceux qui n'ont pas encore répondu, afin de ne pas bloquer la génération du planning.

- Critère : liste des élèves avec statut (soumis / en attente), bouton de relance individuel ou groupé.

User-13 :

En tant qu'élève ou parent, je veux modifier mes vœux d'horaires tant que le planning n'est pas encore généré, afin de corriger une erreur de saisie.

- Critère : édition des vœux possiblement uniquement avant la génération.

User-14 :

En tant que professeur, une fois les vœux de chacun choisis, je veux générer 3 planning en fonction des vœux de mes élèves, afin de choisir la solution la plus adaptée à ma classe.

- Critère : affichage des 3 propositions de planning (tableau hebdomadaire), comparaison possible.

User-15 :

En tant que professeur, je veux choisir si les élèves doivent confirmer leur créneau avant que je valide le planning final, afin d'avoir un processus de validation flexible selon mon contexte.

- Critère : option activable/désactivable par le professeur. Si activée, le planning passe en statut "en attente de la confirmation des élèves" avant la validation finale. Si désactivée, le professeur valide directement.

- Envoie des créneaux à chaque élève.

User-16 :

En tant qu'élève ou parent, je veux être notifié et confirmer mon créneau attribué si mon professeur l'a activé, afin de valider mon horaire de cours.

- Critère : notification envoyée à l'élève avec son créneau, bouton de confirmation affiché uniquement si l'option est activée par le professeur. Affichage du créneau dans tous les cas.

User-17 :

En tant que professeur, je veux un planning global interactif de tous mes élèves (via FullCalendar), afin d'avoir une vision d'ensemble de ma semaine.

- Critère : vue semaine / mois, tous les créneaux affichés avec le nom de l'élève.

User-18 :

En tant que professeur, je veux exporter mon planning vers Google Calendar, afin de l'intégrer directement à mon agenda personnel.

- Critère : bouton d'export, authentification OAuth Google, événement créé dans l'agenda.

User-19 :

En tant qu'élève ou parent, je veux exporter mon planning vers Google Calendar, afin de l'intégrer directement à mon agenda personnel.

- Critère : bouton d'export, authentification OAuth Google, événement créé dans l'agenda.

Should have : Important :

User-20 :

En tant que professeur, je veux modifier manuellement le créneau d'un élève, afin de corriger ou d'ajuster le planning après génération et peut choisir si la modification s'applique sur toute la période, un jour spécifique ou plusieurs jours. .

- Critère : édition du créneau depuis le calendrier.

User-21 :

En tant que professeur, je veux échanger les créneaux de deux élèves, afin de résoudre facilement un conflit d'horaire sans tout recalculer et peut choisir si la modification s'applique sur toute la période, un jour spécifique ou plusieurs jours. .

- Critère : sélection de 2 élèves et confirmer l'échange.

User-22 :

En tant que professeur, je veux cliquer sur le créneau d'un élève depuis mon calendrier pour rédiger les objectifs et conseils pédagogiques après son cours et consulter l'historique des notes précédentes, afin de lui transmettre un suivi personnalisé et suivre sa progression.

- Critère : clic sur un créneau depuis FullCalendar, formulaire texte avec champs objectifs et conseils, historique chronologique des notes précédentes affiché dans le même panneau, sauvegarde en base, notification envoyée automatiquement à l'élève.

User-23 :

En tant qu'élève ou parent, je veux consulter les objectifs et conseils rédigés par mon professeur ou un nouveau planning, afin de préparer mon prochain cours et d'être informé en temps réel.

- Critère : section dédiée accessible depuis le tableau de bord élève, affichage chronologique par créneau.
- Notification push (navigateur) ou in-app.

User-24 :

En tant que professeur, je veux supprimer un créneau, afin de gérer les absences ou arrêts de cours d'un élève et peut choisir si la modification s'applique sur toute la période, un jour spécifique ou plusieurs jours. .

- Critère : suppression avec confirmation, mise à jour du calendrier.

User-25 :

En tant que professeur, je veux créer un événement pour une ou plusieurs de mes classes ou sélectionner des élèves spécifiques (ex : audition, concert, réunion), afin d'informer les personnes concernées d'une date importante.

- Critère : formulaire de création d'événement avec titre, date, heure, description et sélection de la ou des classes concernées et/ou des élèves

spécifiques. Notification envoyée automatiquement aux élèves sélectionnés.
Événement affiché dans le calendrier FullCalendar.

User-26 :

En tant qu'élève ou parent, je veux consulter les événements créés par mon professeur, afin d'être informé des dates importantes.

- Critère : événement affiché dans le calendrier FullCalendar de l'élève, notification reçue lors de la création de l'événement.

Won't have : V2 :

User-27 :

En tant que professeur, je veux encaisser le paiement des cours privés directement depuis l'application, afin de centraliser la gestion administrative.

- Critère : intégration d'un système de paiement en ligne (ex : Stripe), suivi des paiements par élève.

User-28 :

En tant que professeur, je veux envoyer des messages instantanés des vidéos à mes élèves directement depuis l'application, afin de communiquer sans quitter "PlanifPro".

- Critère : messagerie en temps réel et envoi de vidéos entre professeur et élève, notifications de nouveaux messages.

User-29 :

En tant qu'élève ou parent, je veux envoyer des messages instantanés et des vidéos à mon professeur directement depuis l'application, afin de communiquer sans quitter "PlanifPro".

- Critère : messagerie en temps réel entre professeur et élève, envoi de vidéos, notifications de nouveaux messages.

User-30 :

En tant qu'utilisateur, je veux installer l'application sur mon téléphone et accéder à mon planning à tout moment, afin de consulter et gérer mon planning depuis n'importe où.

- Critère : déploiement sur App Store et Google Play via React Native. Pour le MVP, PlanifPro est une PWA installable depuis le navigateur.

I I . Mockups

Charte graphique:

- Couleurs : police, tracer, fond, bouton, 000000 police, tracer, fond, fond, fond navigation, fond card, fond

Fond :

- 4C7AC3
- 0C2863
- 181F72
- 223397
- 4065B6
- 4975BF

Tracer :

- 9095F5
- D5AE13

Police :

- FFFFFFFF

Bouton :

- D59813
- 181F72

Typo logo :

- Konkhmer Sleokchher

Typo titre :

- Libertinus Sans

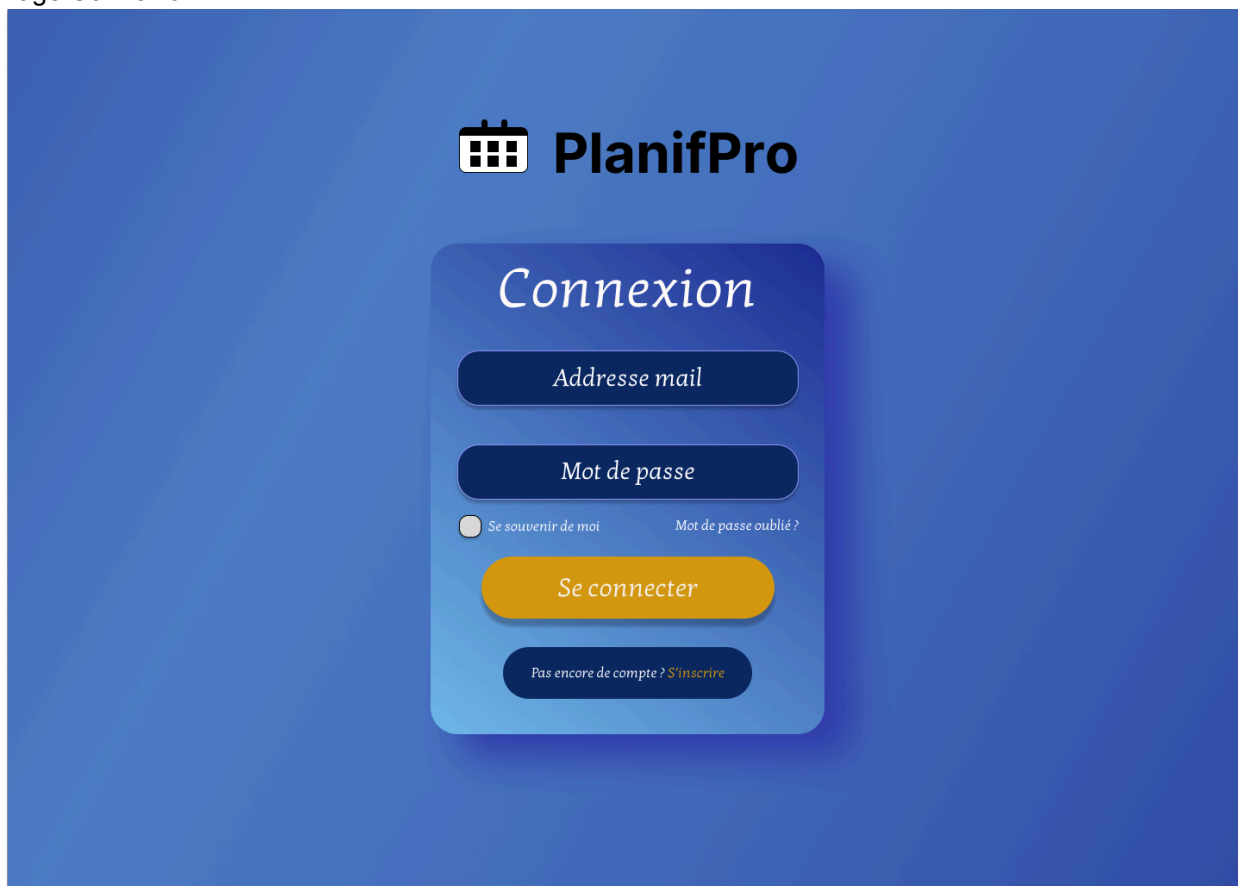
Typo texte de base :

- Kotta One

Liste des écrans:

Professeur / Coach :

1. Page Connexion :



The image shows a login page for 'PlanifPro'. At the top, there is a logo consisting of a calendar icon and the text 'PlanifPro'. Below the logo, the word 'Connexion' is displayed in a large, white, serif font. Underneath 'Connexion', there are two input fields: 'Adresse mail' and 'Mot de passe', both with rounded rectangular borders. Below the 'Mot de passe' field, there is a checkbox labeled 'Se souvenir de moi' and a link 'Mot de passe oublié ?'. A large, orange, rounded rectangular button labeled 'Se connecter' is positioned below these elements. At the bottom of the form, there is a link 'Pas encore de compte ? S'inscrire'.

 **PlanifPro**

Connexion

Adresse mail

Mot de passe

☐ Se souvenir de moi [Mot de passe oublié ?](#)

Se connecter

[Pas encore de compte ? S'inscrire](#)

2. Page Inscription :



PlanifPro

Inscription

Professeur/
Coach

Élèves/
Parent

Prénom

Nom

Adresse mail

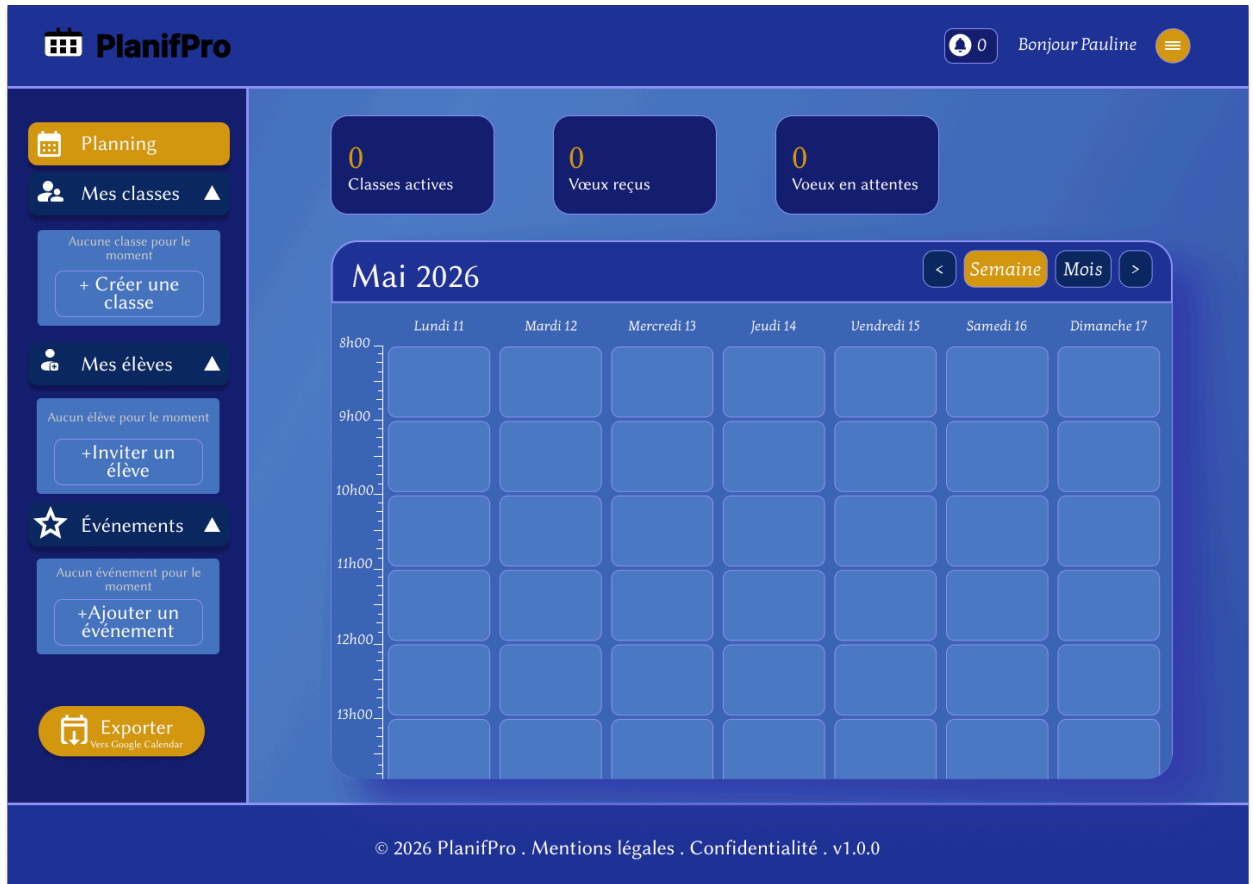
Mot de passe

Confirmer le mot de passe

Créer mon compte

Déjà un compte ? [Se connecter](#)

3. Tableau de bord :



4. Interface mes classes, onglet Élèves :

0

Bonjour Pauline

Planning

Mes classes ▲

Conservatoire

Cours privés

Association

+ Créer une classe

Mes élèves ▲

HC

Hector C.

45 min

PD

Paulin D.

1 heure

GM

Gabriel M.

30 min

CB

Cléo B.

30 min

+ Inviter un élève

Événements ▲

Aucun événement pour le moment

+ Ajouter un événement

Exporter

Vers Google Calendar

Conservatoire

Sep 2026
→ Juin 2027

4 élèves

Mar,
Mer, Jeu

Modifier

+ Inviter un élève

PÉRIODE

01/09/2026 à
30/06/2027

JOURS ET
HORAIRES

Mar 16h - 20h, Mer
15h - 19h30, jeudi
16h - 20h

CONTRAINTES
VOEUX

3 vœux min - 2
jours différents

Code unique de la classe

PAU-4D8M

Copier le code

Inviter par email

Élèves (4)

Vœux

Planning

HC

Hector C.

Durée : 45 min

A rejoint

Modifier

PD

Paulin D.

Durée : 1 heure

A rejoint

Modifier

GM

Gabriel M.

Durée : 30 min

En attente

Modifier

CB

Cléo B.

Durée : non configurée

Pas encore rejoint

Configurer

Prêt à lancer la collecte des vœux ?

2 élèves ont rejoint - 1 en attente - 1 non configuré

Lancer la collecte des vœux

© 2026 PlanifPro . Mentions légales . Confidentialité . v1.0.0

5. Interface mes classes, onglet Voeux :

PlanifPro

0 Bonjour Pauline

Planning

Mes classes

Conservatoire

Cours privés

Association

+ Créer une classe

Mes élèves

HC Hector C.
45 min

PD Paulin D.
1 heure

GM Gabriel M.
30 min

CB Cléo B.
30 min

+ Inviter un élève

Événements

Aucun événement pour le moment

+ Ajouter un événement

Exporter

Vers Google Calendar

Conservatoire

Sep 2026 → Juin 2027

4 élèves

Mar, Mer, Jeu

Modifier

+ Inviter un élève

PÉRIODE

01/09/2026 à 30/06/2027

JOURS ET HORAIRES

Mar 16h - 20h, Mer 15h - 19h30, jeudi 16h - 20h

CONTRAINTES VOEUX

3 vœux min - 2 jours différents

Code unique de la classe

PAU-4D8M

Copier le code

Inviter par email

Élèves (4)

Vœux

Planning

4

Vœux soumis

0

En attente

4

Total élèves

Statut des vœux par élève

Relancer les retardataires

HC Hector C.

Vœux : Mar 16h15 - Mer 17h30 - Jeu 18h00

Soumis

PD Paulin D.

Vœux : Mar 15h00 - Mer 16h30 - Mer 19h00

Soumis

GM Gabriel M.

Vœux : Jeu 17h00 - Jeu 18h30 - Mar 17h15

Soumis

CB Cléo B.

Aucun vœu soumis

En attente

Relancer

Générer les propositions de planning

✓ Hector C.

✓ Paulin D.

✓ Gabriel M.

✗ Cléo B.

Générer (3/4)

⚠ Cléo B. n'a pas soumis ses vœux, elle sera exclue si vous générez maintenant.

© 2026 PlanifPro . Mentions légales . Confidentialité . v1.0.0

6. Interface mes classes, onglet Planning :

PlanifPro

Bonjour Pauline

Planning

Mes classes

Conservatoire

Cours privés

Association

+ Créer une classe

Mes élèves

Hector C.
45 min

Paulin D.
1 heure

Gabriel M.
30 min

Cléo B.
30 min

+ Inviter un élève

Événements

Aucun événement pour le moment

+Ajouter un événement

Exporter
Vers Google Calendar

Conservatoire

Sep 2026 → Juin 2027

4 élèves

Mar, Mer, Jeu

Modifier

+ Inviter un élève

PÉRIODE

01/09/2026 à 30/06/2027

JOURS ET HORAIRES

Mar 16h - 20h, Mer 15h - 19h30, jeudi 16h - 20h

CONTRAINTES VOEUX

3 vœux min - 2 jours différents

Code unique de la classe

PAU-4D8M

Copier le code

Inviter par email

Élèves (4)

Vœux

Planning

4
Vœux soumis

0
En attente

4
Total élèves

Tous les vœux ont été soumis

HC

Hector C.

Vœux : Mar 16h15 - Mer 17h30 - Jeu 18h00

Soumis

PD

Paulin D.

Vœux : Mar 15h00 - Mer 16h30 - Mer 19h00

Soumis

GM

Gabriel M.

Vœux : Jeu 17h00 - Jeu 18h30 - Mar 17h15

Soumis

CB

Cléo B.

Vœux : Mar 16h00 - Jeu 17h45 - Mer 18h15

Soumis

Tous les vœux sont collecté !

Prêt à générer les 3 propositions de planning

Générer les plannings

© 2026 PlanifPro . Mentions légales . Confidentialité . v1.0.0

7. Popup Créer une classe :

Créer une classe

Nom de la classe

Ex : Conservatoire, Cours privés ...

PÉRIODE DES COURS :

Date de début

jj/mm/aaaa

Date de fin

jj/mm/aaaa

JOURS ET HORAIRES DES COURS :

☐ Lundi

☒ Mardi

☒ Mercredi

☐ Jeudi

De

09:00

à

12:00

De

09:00

à

12:00

8. Popup Inviter un élève :

Inviter un élève

✕

Classe concernée

Conservatoire ▼

MÉTHODE D'INVITATION :

Partager le code

Envoyer par email

PAU-4D8M

Code unique de la classe

Copier le code

Partagez ce code à vos élèves pour rejoindre votre classe

Fermer

Envoyer l'invitation

Inviter un élève



Classe concernée

Conservatoire



Conservatoire

Cours privés

Association

PAU-4D8M

Code unique de la classe

Copier le code

Partagez ce code à vos élèves pour rejoindre votre classe

Fermer

Envoyer l'invitation

9. Popup Créer un événement (3 versions) :

Créer un événement

Titre de l'événement

Ex : Audition de piano, Concert de fin d'année ...

Date

jj/mm/aaaa

Heure

12:00

Description

Information complémentaires...

NOTIFIER :

Toutes mes classes

Classes spécifiques

Élèves spécifiques

☒ ● Conservatoire

☐ ● Cours privés

☒ ● Association

Fermer

Envoyer l'invitation

Créer un événement



Titre de l'événement

Ex : Audition de piano, Concert de fin d'année ...

Date

jj/mm/aaaa



Heure

12:00



Description

Information complémentaires...



NOTIFIER :

Toutes mes classes

Classes spécifiques

Élèves spécifiques



● Paulin D.



● Gabriel M.

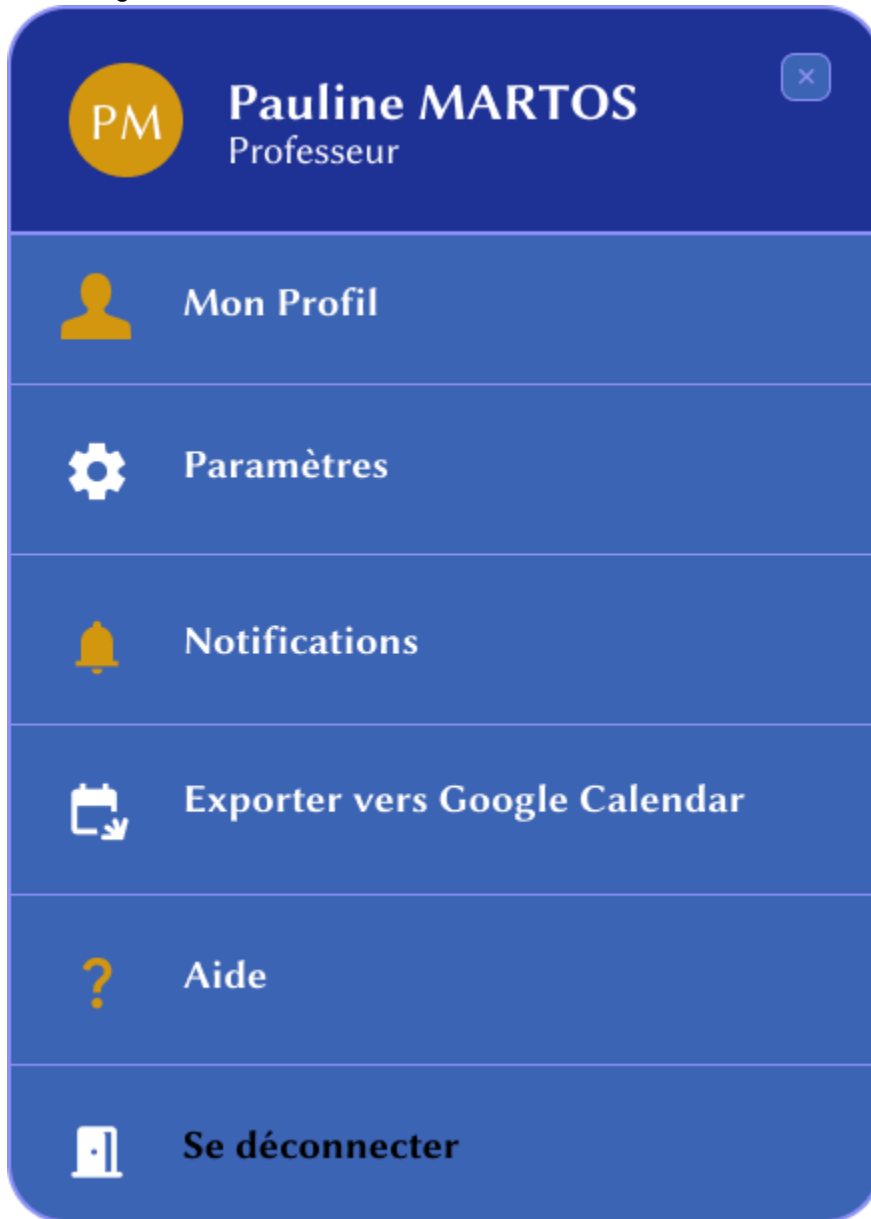


● Cléo B.

Fermer

Envoyer l'invitation

10. Menu burger :

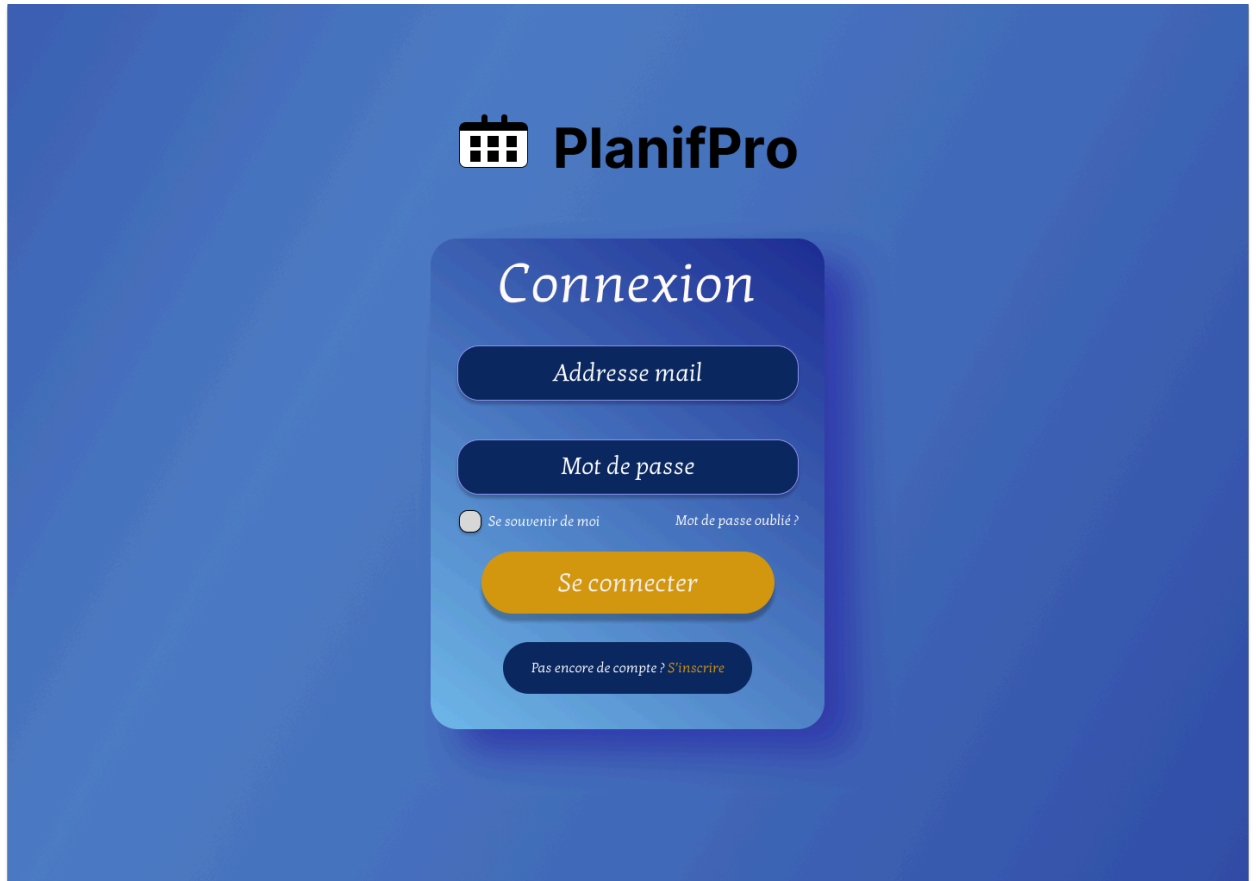


Lien Figma :

<https://www.figma.com/proto/QyXBdfbSOxeJ7fKJpETMdK/PlanifPro?node-id=7-143&t=1zb gEugM4sf0jf7j-1>

Élève / Parent :

1. Connexion :



The image shows a login interface for PlanifPro. At the top, there is a logo consisting of a calendar icon and the text "PlanifPro". Below the logo, the word "Connexion" is displayed in a large, white, serif font. Underneath "Connexion", there are two input fields: "Adresse mail" and "Mot de passe", both with rounded rectangular borders. Below the "Mot de passe" field, there is a checkbox labeled "Se souvenir de moi" and a link "Mot de passe oublié ?". A large, orange, rounded rectangular button labeled "Se connecter" is positioned below these elements. At the bottom of the login form, there is a link "Pas encore de compte ? S'inscrire" in a smaller font.

 **PlanifPro**

Connexion

Adresse mail

Mot de passe

☐ Se souvenir de moi [Mot de passe oublié ?](#)

Se connecter

Pas encore de compte ? [S'inscrire](#)

2. Inscription :



PlanifPro

Inscription

Professeur/
Coach

Élèves/
Parent

Prénom

Nom

Adresse mail

Mot de passe

Confirmer le mot de passe

Créer mon compte

Déjà un compte ? [Se connecter](#)

3. Tableau de Bord Élève / Parent :

 **PlanifPro**

 0 Bonjour Simon 

 **Planning**

 **Mes Objectifs** ▲
Aucun objectifs pour le moment

 **Mes Professeurs** ▲
Aucun Professeur pour le moment
[+Ajouter le code](#)

 **Événements** ▲
Aucun événement pour le moment
[+Accepter un événement](#)

 **Exporter**
Vers Google Calendar

 **Rejoindre une classe**
Entrer un code de classe

 **Mes vœux**
Soumettre mes disponibilités

 **Exporter**
Vers Google Calendar

 Pauline MARTOS a envoyé le formulaire de vœux [✓ Soumettre vos vœux](#)

 **Piano Débutant : Mercredi 10h00**
Professeur Pauline MARTOS : 30 min [✓ Ajouter au calendrier](#)

Mai 2026  **Semaine** **Mois** 

	Lundi 11	Mardi 12	Mercredi 13	Jeudi 14	Vendredi 15	Samedi 16	Dimanche 17
8h00		 Ajouter vos cours					
9h00							
10h00							
11h00							
12h00							
13h00							

© 2026 PlanifPro . Mentions légales . Confidentialité . v1.0.0

4. Popup Formulaire des vœux :

Soumettre mes vœux : Conservatoire

3 vœux obligatoire

Choisir 2 jours minimum

Cochez vos jours disponibles et choisissez vos créneaux

☐ Mardi

0 créneaux

☒ Mercredi

2 créneaux

15h00

15h15

15h30

15h45

16h00

16h15

16h30

16h45

17h00

17h15

17h30

17h45

18h00

18h15

18h30

18h45

19h00

19h15

19h30

19h45

☐ Jeudi

0 créneaux

Vœux sélectionnés :

Vœu 1 : Mer 16h15

Vœu 2 : Mer 19h00

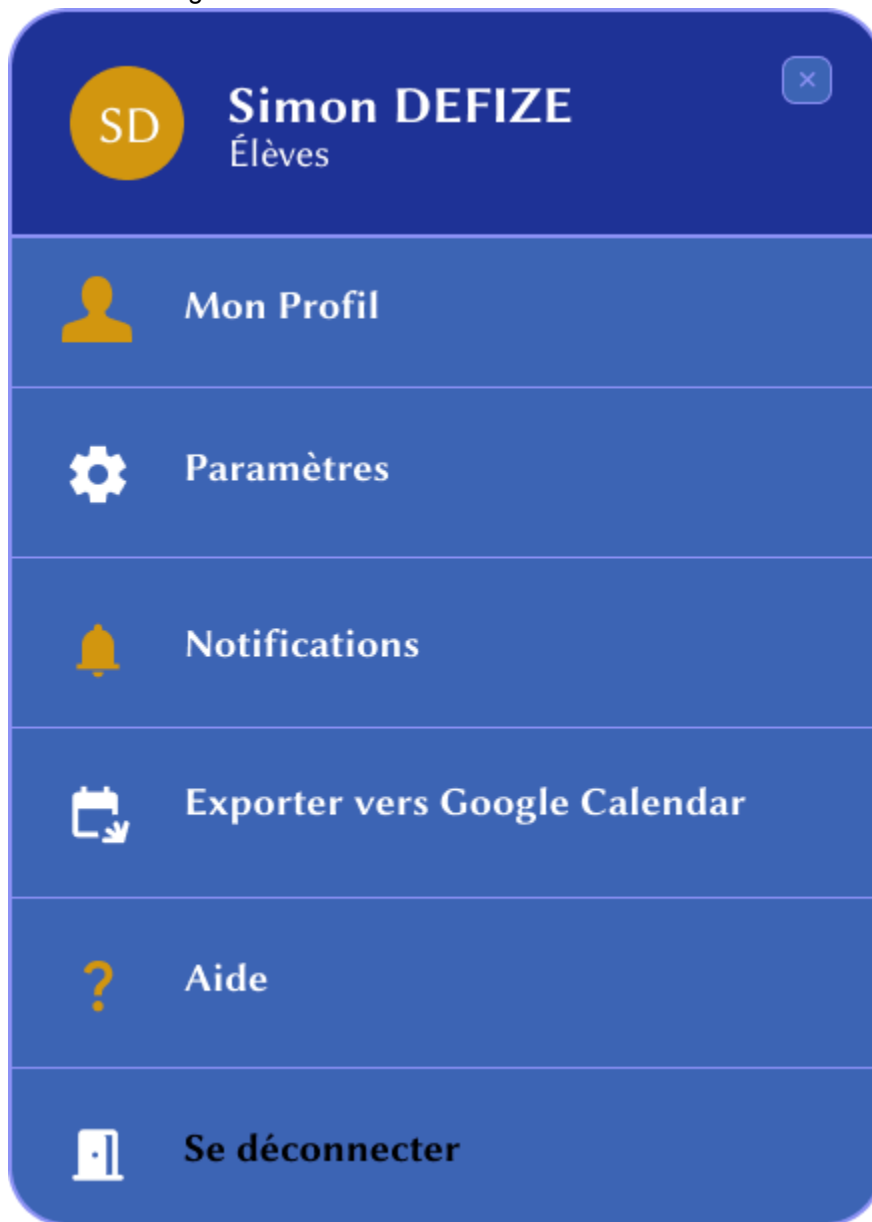
Vœu 3 : non sélectionné

Vœux : 2 / 3

Annuler

Soumettre →

5. Menu burger :



Lien Figma :

<https://www.figma.com/proto/QyXBdfbSOxeJ7fKJpETMdK/PlanifPro?node-id=7-143&t=1zb gEugM4sf0jf7j-1>

I I I . Architecture système :

1. Les Utilisateurs :

“PlanifPro” s'adresse à deux types d'utilisateurs. Le Professeur/Coach accède à l'application depuis son navigateur sur ordinateur ou mobile pour gérer ses classes, configurer ses disponibilités, consulter les propositions de planning et assurer le suivi pédagogique de ses élèves. L'Élève/Parent accède à l'application depuis son navigateur pour rejoindre une classe, soumettre ses vœux d'horaires, consulter son créneau attribué et recevoir les objectifs de son professeur. Les deux types d'utilisateurs reçoivent des notifications push via Firebase Cloud Messaging (FCM).

2. Frontend : React PWA (Vercel) :

Le frontend est développé en React sous forme de Progressive Web App (PWA). Il est déployé sur Vercel qui assure un déploiement automatique depuis GitHub. La PWA permet à l'application d'être installée sur l'écran d'accueil du téléphone. Le Service Worker (script JavaScript qui tourne en arrière-plan dans le navigateur, même quand l'application est fermée) est le composant clé de la PWA, il assure trois rôles essentiels :

- La mise en cache des ressources pour le mode hors-ligne
- La réception des notifications push via FCM
- L'installation de l'application sur mobile.

Le calendrier interactif est géré par FullCalendar et le style par Tailwind CSS.

Le frontend communique avec le backend via des appels REST API (interface de communication entre le frontend et le backend) en HTTPS et reçoit des réponses au format JSON (format léger d'échange de données).

Il est composé de quatre modules principaux :

- **Tableau de bord Professeur** : gestion des classes, planning global, suivi pédagogique
- **Tableau de bord Élève** : consultation du créneau, objectifs, événements
- **Gestion des classes** : création, configuration, collecte des vœux, génération du planning
- **Formulaire des vœux** : sélection des jours et créneaux disponibles par l'élève

3. Backend : Python / Flask (Render) :

Le backend est développé en Python avec le framework Flask.

Il est déployé sur Render avec la base de données PostgreSQL.

Il expose une API REST (interface de communication entre le frontend et le backend) consommée par le frontend, il reçoit les requêtes HTTP et renvoie des réponses au format JSON.

Il intègre le SDK (Software Development Kit : kit d'outils tout prêt fourni par un service pour l'intégrer facilement dans une application) Python de SendGrid pour les emails et le SDK Firebase Admin pour les notifications push.

Il est composé de quatre modules principaux :

- Auth (JWT) : La gestion de l'inscription, connexion et déconnexion sécurisée via JSON Web Tokens (JWT : Le système de tokens sécurisés pour authentifier les utilisateurs). Le token est généré à la connexion et vérifié à chaque requête pour sécuriser les échanges entre le frontend et le backend.
- Algorithme de planning, c'est le cœur technique de l'application. Il génère 3 propositions de planning en croisant les disponibilités du professeur et les vœux des élèves, en tenant compte des durées individuelles de cours.
- Suivi pédagogique : Permet au professeur de rédiger objectifs et conseils par créneau. L'élève reçoit une notification et peut les consulter depuis son tableau de bord.
- Notifications / Emails : Gère l'envoi des notifications push via le SDK Firebase Admin et des emails d'invitation via le SDK Python de SendGrid.

4. Base de données :

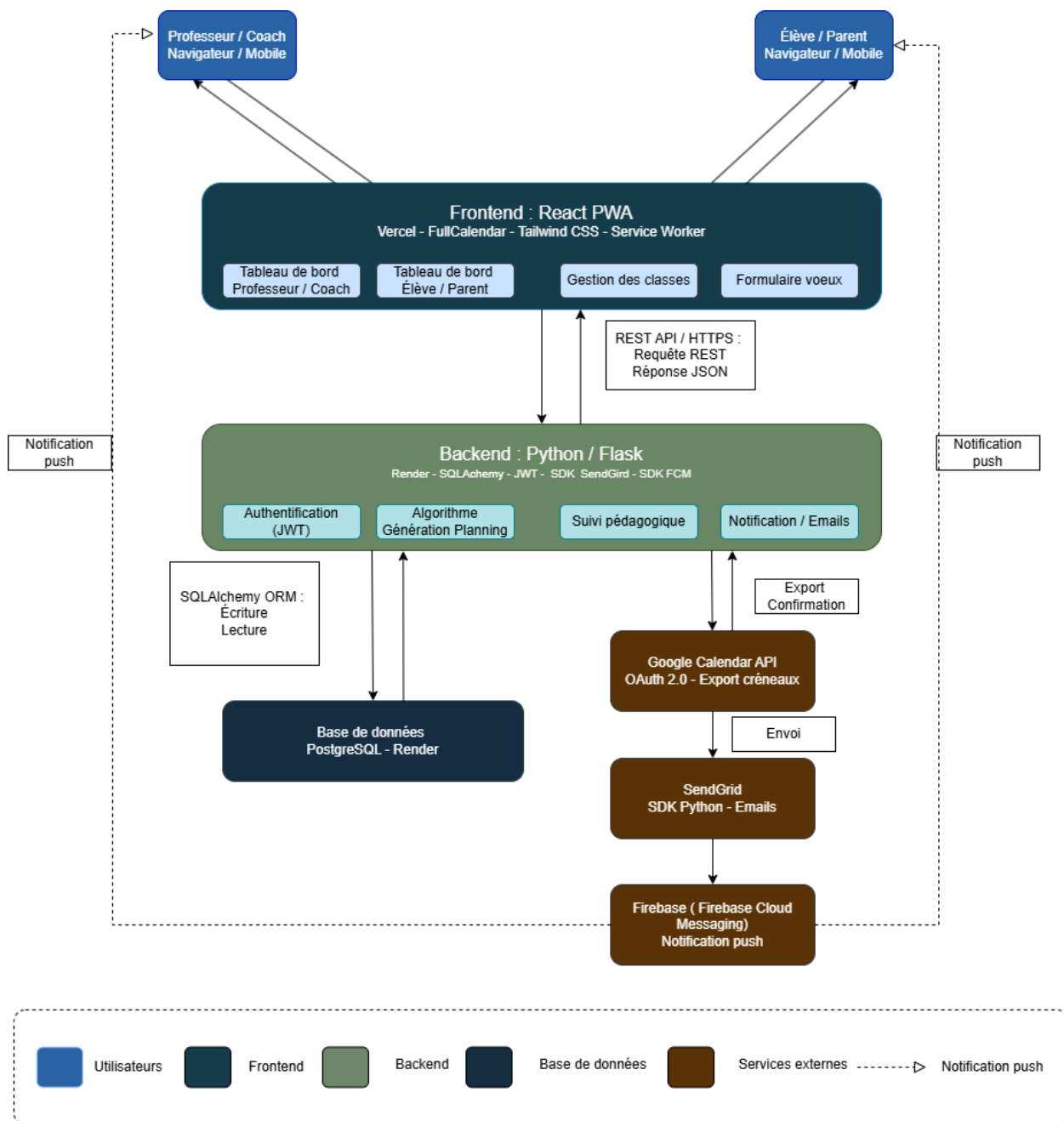
PostgreSQL (Render) La base de données est PostgreSQL, hébergée sur Render au même endroit que le backend. Elle est pilotée par l'ORM SQLAlchemy (ORM : Object Relational Mapper est un outil qui permet de manipuler les données en Python sans écrire de SQL brut). Le backend lit et écrit dans la base de données à chaque requête. Elle stocke l'ensemble des données de l'application : utilisateurs, classes, élèves, vœux, créneaux, plannings, objectifs pédagogiques, tokens FCM et notifications.

5. Services externes :

- Google Calendar API (OAuth 2.0) : Permet aux professeurs et aux élèves d'exporter leurs créneaux directement vers leur agenda Google Calendar. L'authentification se fait via le protocole OAuth 2.0 (protocole d'autorisation sécurisé qui permet à une application d'accéder à un service tiers sans stocker le mot de passe de l'utilisateur). Le backend envoie les créneaux à Google Calendar qui confirme l'export.

- SendGrid : Le service d'envoi d'emails utilisé pour envoyer les invitations de classe aux élèves avec le code de classe, ainsi que les relances pour la soumission des vœux. Intégré via le SDK Python officiel de SendGrid (Software Development Kit est un kit d'outils tout prêt pour utiliser SendGrid en Python). Le flux est unidirectionnel, le backend envoie, SendGrid délivre.
 -
- Firebase Cloud Messaging (FCM) est un service de Google utilisé pour envoyer des notifications push fiables au professeur et aux élèves sur tous les navigateurs et appareils, même quand l'application est fermée. Lorsqu'un utilisateur autorise les notifications, un token FCM (identifiant unique généré par le navigateur pour identifier l'appareil de l'utilisateur) est sauvegardé en base de données. Le backend utilise le SDK Firebase Admin (Software Development Kit est un kit d'outils tout prêt pour utiliser Firebase en Python) pour envoyer des notifications à FCM, qui les délivre ensuite aux appareils des utilisateurs concernés.

Diagramme de l'Architecture système PlanifPro :



IV. Composants, Classes et Base de données

Diagramme de classes

Le diagramme de classes ci-dessous représente la structure principale de l'application "PlanifPro". Il permet de modéliser les différentes entités métier du système, leurs attributs, leurs méthodes ainsi que les relations entre elles.

Ce diagramme constitue la base de conception du backend de l'application.

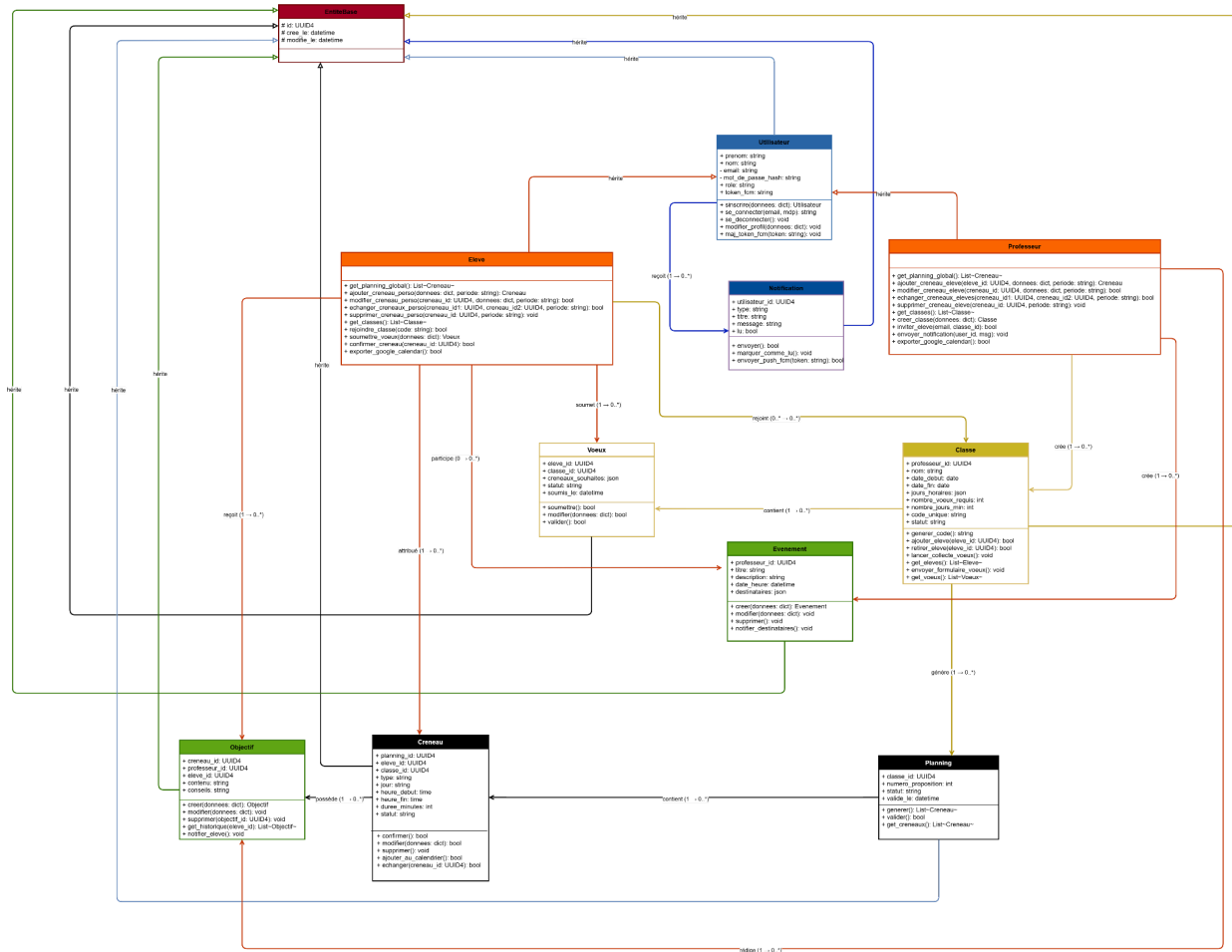
Il met en évidence les différents rôles utilisateurs (professeur, élève), la gestion des classes, des plannings, des vœux, des notifications et du suivi pédagogique.

L'objectif de cette modélisation est de :

- Structurer les données de manière cohérente
- Préparer la conception de la base de données relationnelle
- Faciliter l'implémentation des fonctionnalités métier
- Visualiser les interactions entre les composants principaux du système

Le diagramme a été conçu selon une approche orientée objet afin de garantir une architecture évolutive, maintenable et adaptée aux besoins fonctionnels de l'application.

Classes Backend :



Relations entre classe :

1. Professeur $\rightarrow$ Classe :
 - Cardinalité :
1 Professeur $\rightarrow$ 0..* Classes
 - Explication :
Un professeur peut créer et gérer plusieurs classes.
Une classe appartient à un seul professeur.

2. Élève ↔ Classe :
 - Cardinalité :
0..* Élèves ↔ 0..* Classes
 - Explication :
Un élève peut rejoindre plusieurs classes.
Une classe peut contenir plusieurs élèves.
3. Classe → Vœux :
 - Cardinalité :
1 Classe → 0..* Vœux
 - Explication :
Une classe peut contenir plusieurs vœux soumis par les élèves.
Chaque vœu est associé à une seule classe.
4. Élève → Vœux :
 - Cardinalité :
1 Élève → 0..* Vœux
 - Explication :
Un élève peut soumettre plusieurs vœux selon ses différentes classes.
Chaque vœu appartient à un seul élève.
5. Classe → Planning :
 - Cardinalité :
1 Classe → 0..* Plannings
 - Explication :
Une classe peut générer plusieurs propositions de planning.
Chaque planning appartient à une seule classe.
6. Planning → Créneau :
 - Cardinalité :
1 Planning → 1..* Créneaux
 - Explication :
Un planning est composé de plusieurs créneaux horaires.
Chaque créneau appartient à un seul planning.
7. Élève → Créneau :
 - Cardinalité :
1 Élève → 0..* Créneaux
 - Explication :
Un élève peut avoir plusieurs créneaux de cours.
Chaque créneau est attribué à un seul élève.

8. Professeur → Créneau :

- Cardinalité :

1 Professeur → 0..* Créneaux

- Explication :

Un professeur peut gérer plusieurs créneaux dans ses différents plannings.
Chaque créneau est associé à un seul professeur.

9. Créneau → Objectif :

- Cardinalité :

1 Créneau → 0..1 Objectif

- Explication :

Un créneau peut posséder un objectif pédagogique et des conseils associés.
Chaque objectif appartient à un seul créneau.

10. Utilisateur → Notification :

- Cardinalité :

1 Utilisateur → 0..* Notifications

- Explication :

Un utilisateur peut recevoir plusieurs notifications.
Chaque notification appartient à un seul utilisateur.

11. Professeur → Événement :

- Cardinalité :

1 Professeur → 0..* Événements

- Explication :

Un professeur peut créer plusieurs événements.
Chaque événement est créé par un seul professeur.

12. Événement ↔ Élève :

- Cardinalité :

0..* Événements ↔ 0..* Élèves

- Explication :

Un événement peut concerner plusieurs élèves.
Un élève peut participer à plusieurs événements.

13. Événement ↔ Classe :

- Cardinalité :
1 Événement ↔ 0..* Classes
- Explication :
Un événement peut être destiné :
 - à une classe entière,
 - à plusieurs classes.

Liste des classes :

Classe EntiteBase

Attributs

- # id: UUID4
- # cree_le: datetime
- # modifie_le: datetime

Classe Utilisateur

Attributs

- + prenom: string
- + nom: string
- - email: string
- - mot_de_passe_hash: string
- + role: string
- + token_fcm: string

Méthodes

- + sinscrire(donnees: dict): Utilisateur
- + se_connecter(email: string, mdp: string): string
- + se_deconnecter(): void
- + modifier_profil(donnees: dict): void
- + maj_token_fcm(token: string): void

Classe Professeur

Méthodes

- + get_planning_global(): List~Creneau~
- + ajouter_creneau_eleve(eleve_id: UUID4, donnees: dict, periode: string): Creneau
- + modifier_creneau_eleve(creneau_id: UUID4, donnees: dict, periode: string): bool
- + supprimer_creneau_eleve(creneau_id: UUID4, periode: string): void
- + echanger_creneaux_eleves(creneau_id1: UUID4, creneau_id2: UUID4, periode: string): bool
- + get_classes(): List~Classe~
- + creer_classe(donnees: dict): Classe
- + inviter_eleve(email: string, classe_id: UUID4): bool
- + envoyer_notification(user_id: UUID4, msg: string): void

- + exporter_google_calendar(): bool

Classe Eleve

Méthodes

- + get_planning_global(): List~Creneau~
- + ajouter_creneau_perso(donnees: dict, periode: string): Creneau
- + modifier_creneau_perso(creneau_id: UUID4, donnees: dict, periode: string): bool
- + supprimer_creneau_perso(creneau_id: UUID4, periode: string): void
- + echanger_creneaux_perso(creneau_id1: UUID4, creneau_id2: UUID4, periode: string): bool
- + get_classes(): List~Classe~
- + rejoindre_classe(code: string): bool
- + soumettre_voeux(donnees: dict): Voeux
- + confirmer_creneau(creneau_id: UUID4): bool
- + exporter_google_calendar(): bool

Classe Classe

Attributs

- + professeur_id: UUID4
- + nom: string
- + date_debut: date
- + date_fin: date
- + jours_horaires: json
- + nb_voeux_requis: int
- + nb_jours_min: int
- + code_unique: string
- + statut: string

Méthodes

- + generer_code(): string
- + ajouter_eleve(eleve_id: UUID4): bool
- + retirer_eleve(eleve_id: UUID4): bool
- + lancer_collecte_voeux(): void
- + envoyer_formulaire_voeux(): void
- + get_eleves(): List~Eleve~
- + get_voeux(): List~Voeux~

Classe Voeux

Attributs

- + eleve_id: UUID4
- + classe_id: UUID4
- + creneaux_souhaitees: json
- + statut: string
- + soumis_le: datetime

Méthodes

- + soumettre(): bool
- + modifier(donnees: dict): bool
- + valider(): bool

Classe Planning

Attributs

- + classe_id: UUID4
- + numero_proposition: int
- + statut: string
- + valide_le: datetime

Méthodes

- + generer(): List~Creneau~
- + valider(): bool

Classe Creneau

Attributs

- + planning_id: UUID4
- + eleve_id: UUID4
- + classe_id: UUID4
- + type: string
- + jour: string
- + heure_debut: time
- + heure_fin: time
- + duree_minutes: int
- + statut: string

Méthodes

- + confirmer(): bool
- + modifier(donnees: dict): bool
- + supprimer(): void
- + echanger(creneau_id: UUID4): bool
- + ajouter_au_calendrier(): bool

Classe Objectif

Attributs

- + creneau_id: UUID4
- + professeur_id: UUID4
- + eleve_id: UUID4
- + contenu: string
- + conseils: string

Méthodes

- + creer(donnees: dict): Objectif
- + modifier(donnees: dict): void
- + supprimer(objectif_id: UUID4): void
- + get_historique(eleve_id: UUID4): List~Objectif~

- + notifier_eleve(): void

Classe Evenement

Attributs

- + professeur_id: UUID4
- + titre: string
- + description: string
- + date_heure: datetime
- + destinataires: json

Méthodes

- + creer(donnees: dict): Evenement
- + modifier(donnees: dict): void
- + supprimer(): void
- + retirer_eleve(eleve_id: UUID4): bool
- + notifier_destinataires(): void

Classe Notification

Attributs

- + utilisateur_id: UUID4
- + type: string
- + titre: string
- + message: string
- + lu: bool

Méthodes

- + envoyer(): bool
- + marquer_comme_lu(): void
- + envoyer_push_fcm(token: string): bool

Diagramme Entité-Relation (ERD)

Le diagramme Entité-Relation (ERD) représente la structure de la base de données PostgreSQL de PlanifPro.

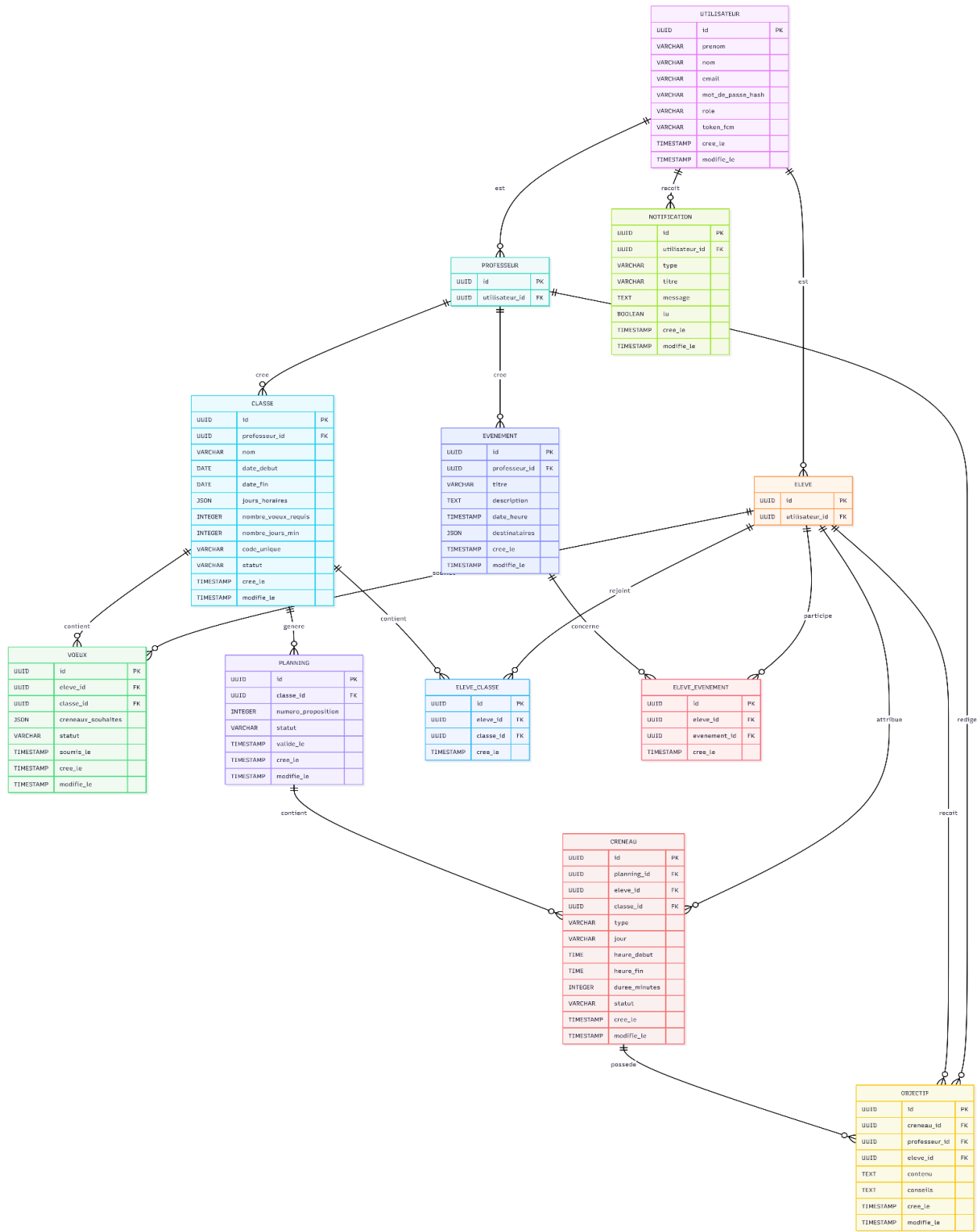
Il modélise l'ensemble des tables, leurs colonnes avec leurs types de données, ainsi que les relations entre elles.

La base de données est pilotée par l'ORM SQLAlchemy qui permet de manipuler les données en Python sans écrire de SQL brut. Chaque table possède un identifiant unique de type UUID (Universally Unique Identifier : identifiant universel unique généré aléatoirement) ainsi que deux colonnes de traçabilité : `cree_le` et `modifie_le` qui enregistrent automatiquement les dates de création et de modification de chaque enregistrement.

Les relations **many-to-many** (plusieurs à plusieurs) entre les tables sont gérées par des tables de liaison :

- `ELEVE_CLASSE` → liaison entre un élève et une classe
- `ELEVE_EVENEMENT` → liaison entre un élève et un événement

Schéma de Base de données (Entity Relationship Diagram) :



Composants Frontend

Le frontend de PlanifPro est développé en React sous forme de PWA (Progressive Web App), déployée sur Vercel. Le style est géré par Tailwind CSS et le calendrier interactif par FullCalendar.

L'interface est organisée en 5 pages principales : **PageConnexion**, **PageInscription**, **DashboardProfesseur**, **DashboardEleve** et **InterfaceClasse**. Chaque page est découpée en composants React indépendants et réutilisables.

Chaque composant est documenté selon quatre axes :

- Les **props** qu'il reçoit de son composant parent
- Le **state** qu'il gère en interne
- Les **composants enfants** qu'il orchestre
- Les **appels API** qu'il effectue vers le backend Flask

Cette architecture orientée composants garantit une séparation claire des responsabilités, facilite la maintenance et permet une montée en charge progressive de l'interface au fil du développement.

Composant	Rôle	Props	State	Composants enfants	API / Redirection
Page Connexion					
Page Connexion	Permet à l'utilisateur de se connecter à PlanifPro	aucune	email: string motDePasse: string erreur: string chargement: boo	FormulaireConnexion BoutonConnexion LienInscription	POST /auth/login -> DashboardProfesseur -> DashboardEleve
Page Inscription					
Page Inscription	Permet à l'utilisateur de créer un compte Professeur ou Élève	aucune	prenom: string nom: string email: string motDePasse: string confirmation: string role: string erreur: string chargement: bool	FormulaireInscription SelecteurRole BoutonCreerCompte LienConnexion	POST /auth/register -> DashboardProfesseur -> DashboardEleve
Tableau de bord Professeur					
Tableau de bord Professeur	Vue principale du professeur avec calendrier global et gestion des classes	utilisateur: object	classes: array creneaux: array notifications: int vueCalendrier: string	Navbar SidebarProfesseur CalendrierFullCalendar StatsCards PopupCréerClasse PopupInviterEleve PopupCreerEvenement MenuBurg	GET /classes GET /creneaux GET /notifications
Tableau de bord Élève					
Tableau de bord Élève	Vue principale de l'élève avec calendrier pe	utilisateur: object	classes: array creneaux: array objectifs: array notifications: int vueCalendrier: string	Navbar SidebarEleve CalendrierFullCalendar CarteNotification CarteCreneauAttribue PopupVoeux MenuBurger Footer	GET /classes GET /creneaux GET /objectifs GET /notifications
Interface Classes					
Interface Classes	Gestion	classId: UUID	classe: object	OngletEleves	GET /classes/:id

	détaillée d'une classe - onglets Élèves, Voeux, Planning	utilisateur: object	eleves: array voeux: array plannings: array ongletActif: strin	OngletVoeux OngletPlanning CarteClasse ListeEleves StatutVoeux PropositionsPlan ning CalendrierDragDr op BanniereGenerer BanniereValider	GET /voeux POST/planning/gene rer PUT /planning/valider
--	---	------------------------	---	--	---

V. Diagrammes de séquence

Les diagrammes de séquence décrivent les interactions entre les composants de PlanifPro pour les cas d'utilisation critiques de l'application. Ils montrent l'ordre chronologique des échanges entre l'utilisateur, le frontend React PWA, le backend Flask et la base de données PostgreSQL, en précisant les messages envoyés, les codes de réponse HTTP et les cas d'erreur gérés.

Trois cas d'utilisation ont été retenus comme représentatifs de l'ensemble de l'architecture :

- **Inscription / Connexion** : le flux d'authentification JWT, commun aux deux types d'utilisateurs (Professeur et Élève/Parent)
- **Soumission des vœux et génération du planning** : le cœur fonctionnel de l'application, impliquant l'algorithme de génération et la validation par le professeur
- **Suivi pédagogique et notification FCM** : le flux post-cours, du dépôt des objectifs par le professeur jusqu'à la réception de la notification push par l'élève

Chaque diagramme suit la notation UML standard avec des fragments pour représenter les chemins alternatifs (erreurs, cas limites) et les codes HTTP associés à chaque réponse.

Diagramme de séquence : Inscription / connexion :

Diagramme 1 : Inscription / Connexion :

Ce diagramme illustre le flux d'authentification commun aux deux types d'utilisateurs de PlanifPro. Il couvre la création d'un compte avec validation du mot de passe, détection des doublons d'email et hashage bcrypt, ainsi que la connexion avec vérification du mot de passe et génération du JWT token.

Chaque cas d'erreur est traité explicitement :

- 400 pour les données invalides
- 409 en cas de conflit d'email
- 401 pour un mot de passe incorrect
- 404 si l'utilisateur est introuvable
- 500 en cas d'erreur base de données.

Diagramme de séquence : Inscription

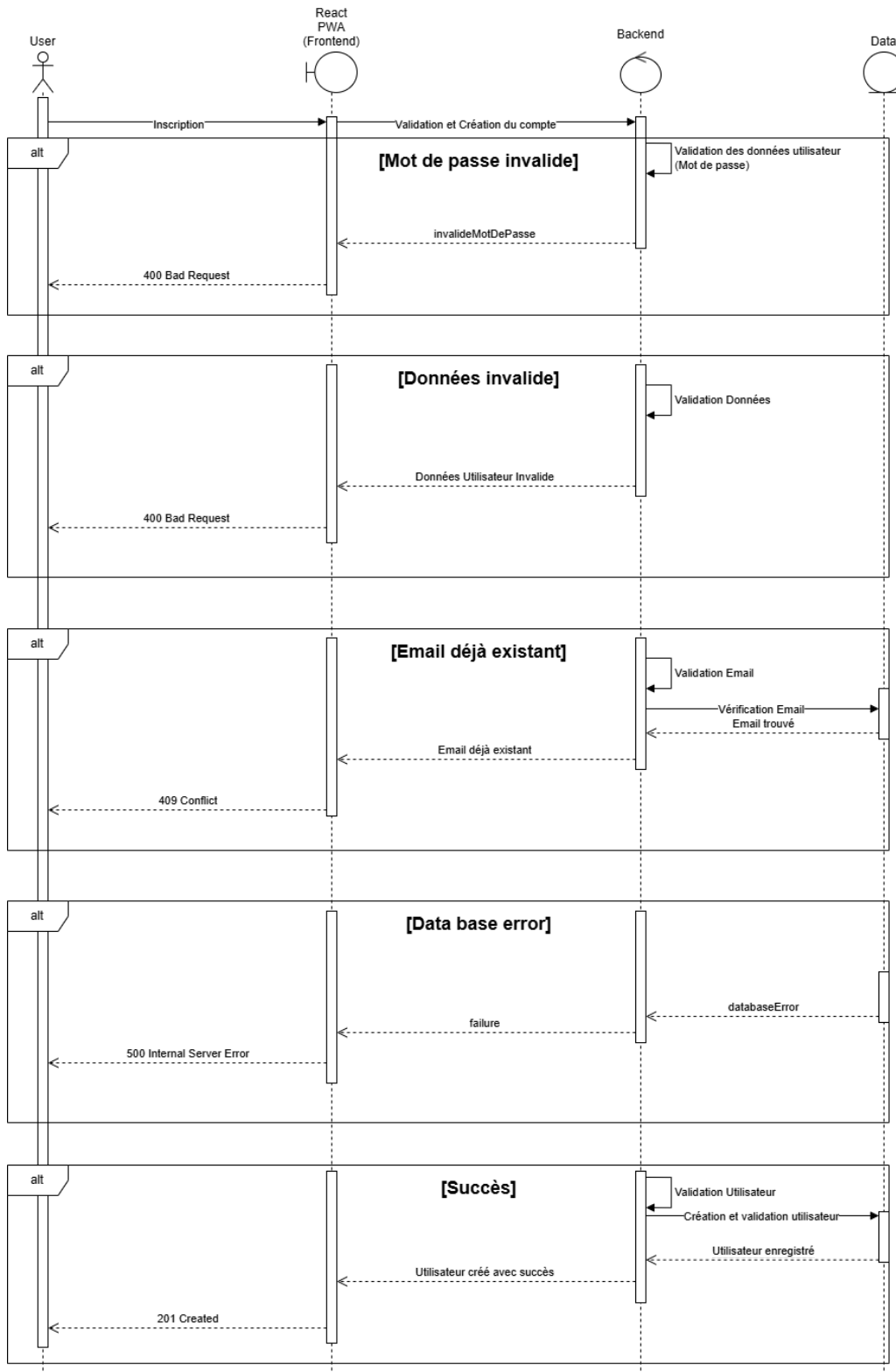


Diagramme de séquence : Connexion

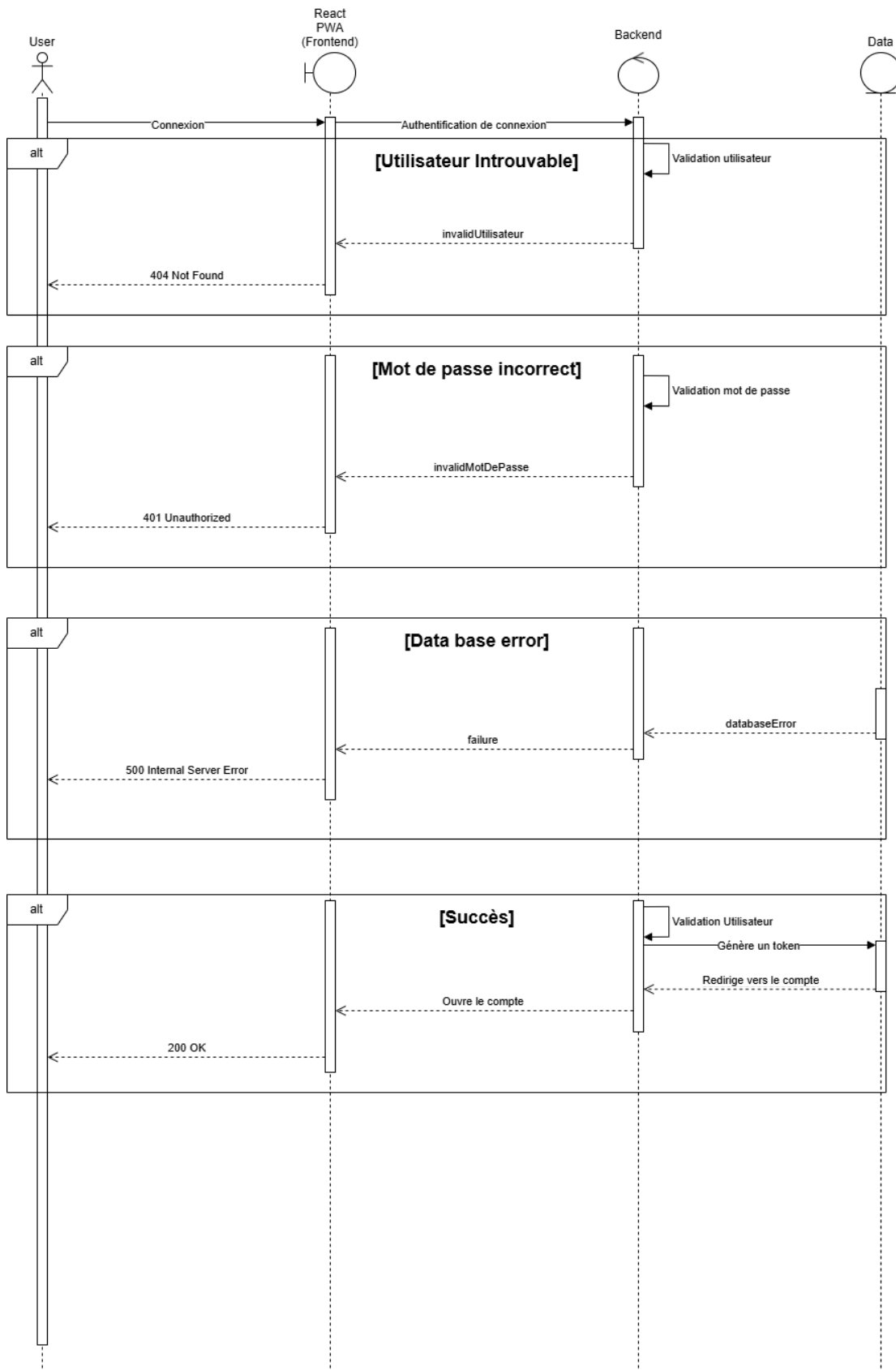


Diagramme de séquence : Collecte et Soumission des vœux et génération des 3 propositions de planning et validation par le professeur :

Diagramme de séquence : Collecte des vœux :

Diagramme 2 : Soumission des vœux et génération du planning :

Ce diagramme est le cœur fonctionnel de PlanifPro. Il se décompose en deux parties. La partie A couvre le lancement de la collecte des vœux par le professeur, l'envoi de la notification FCM aux élèves, et la soumission des vœux par chaque élève avec gestion des erreurs:

- 400 (vœux insuffisants)
- 409 (planning déjà généré)
- 500 (erreur base de données)

La partie B couvre la génération des trois propositions de planning par l'algorithme Flask, la gestion de l'erreur :

- 500 en cas d'échec de l'algorithme
- puis la validation du planning par le professeur avec attribution des créneaux et notification FCM à chaque élève.

Diagramme de séquence : Collecte des vœux

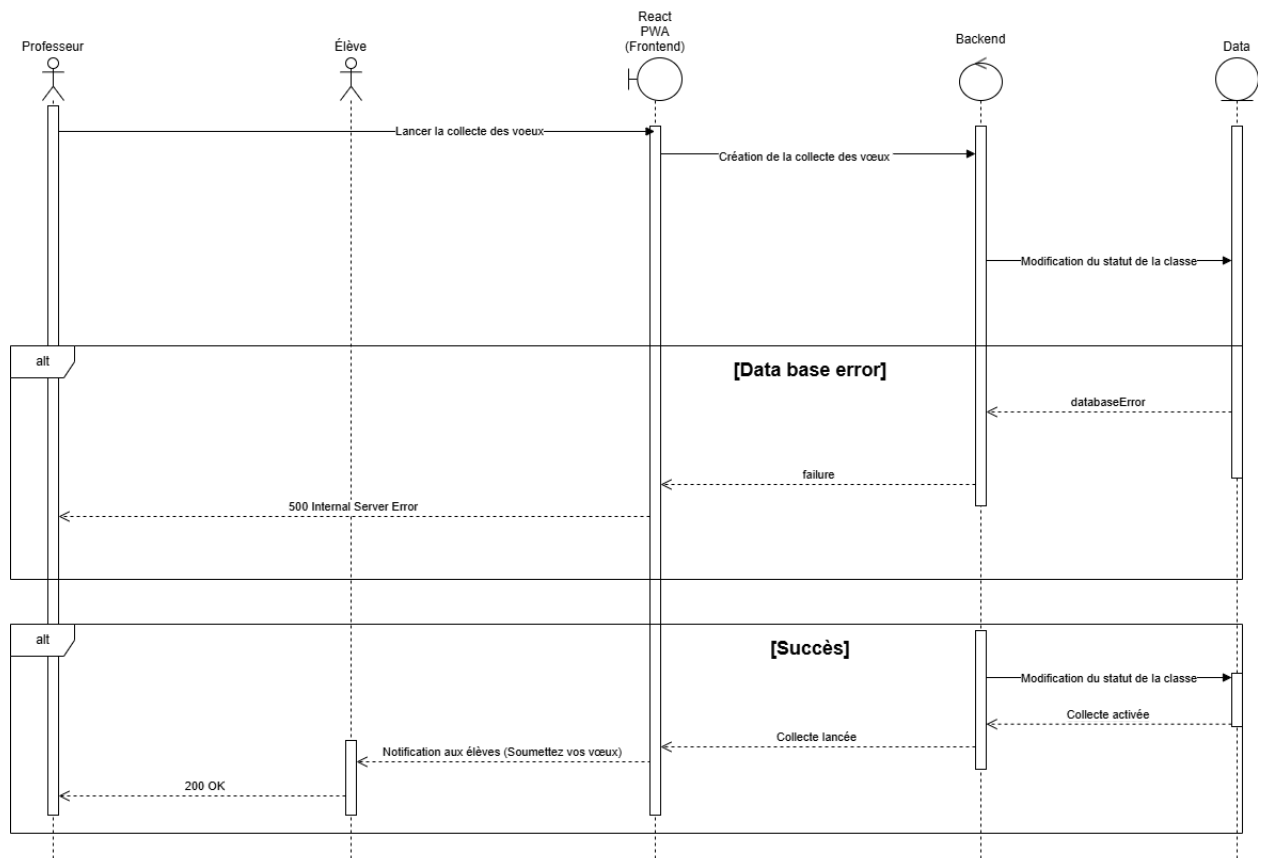


Diagramme de séquence : Soumission des vœux :

Diagramme de séquence : Soumission des vœux

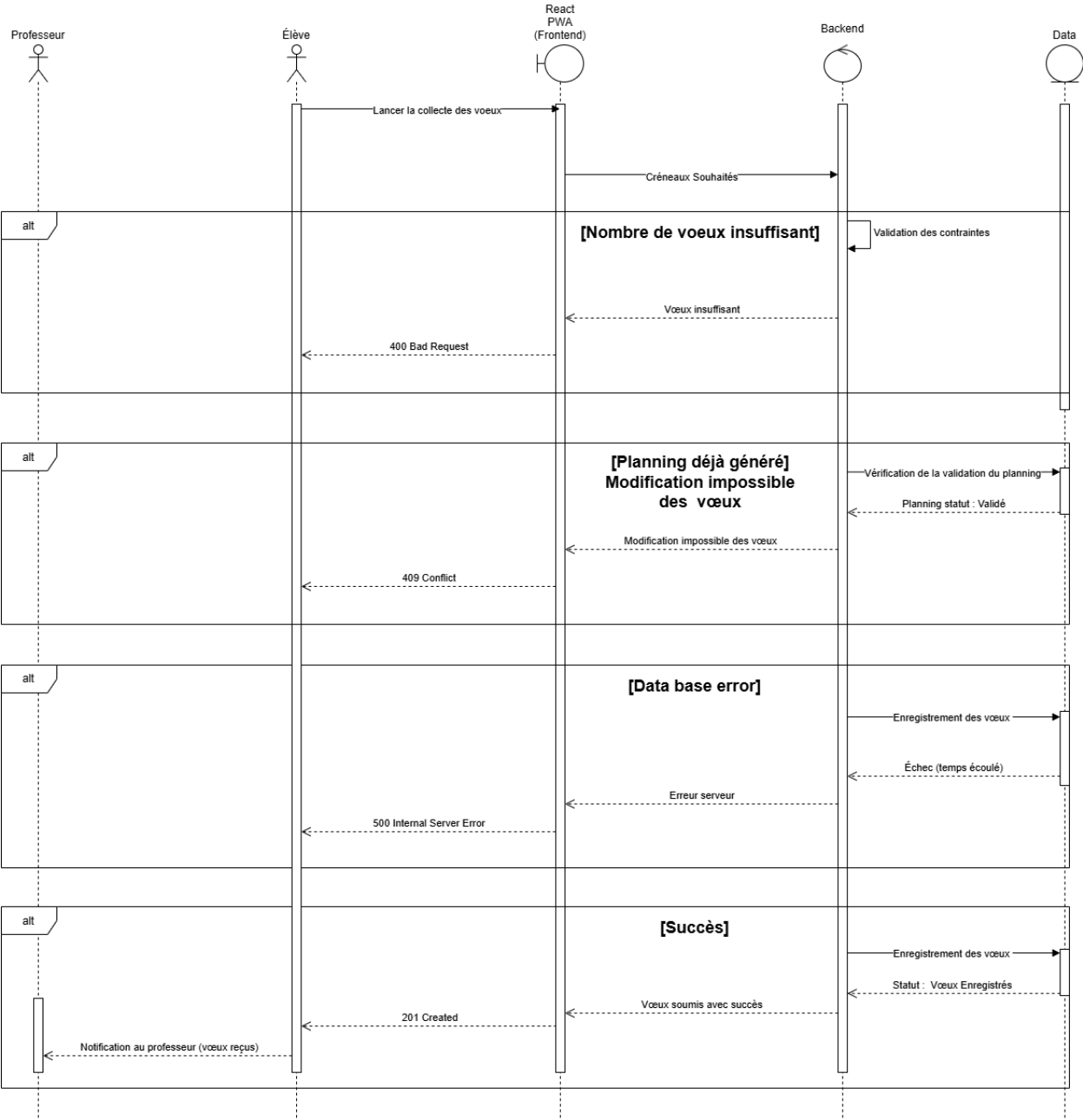


Diagramme de séquence : génération des 3 propositions de planning et validation par le professeur :

Diagramme de séquence : Génération des 3 propositions de planning :

Diagramme de séquence : Génération des 3 propositions de planning

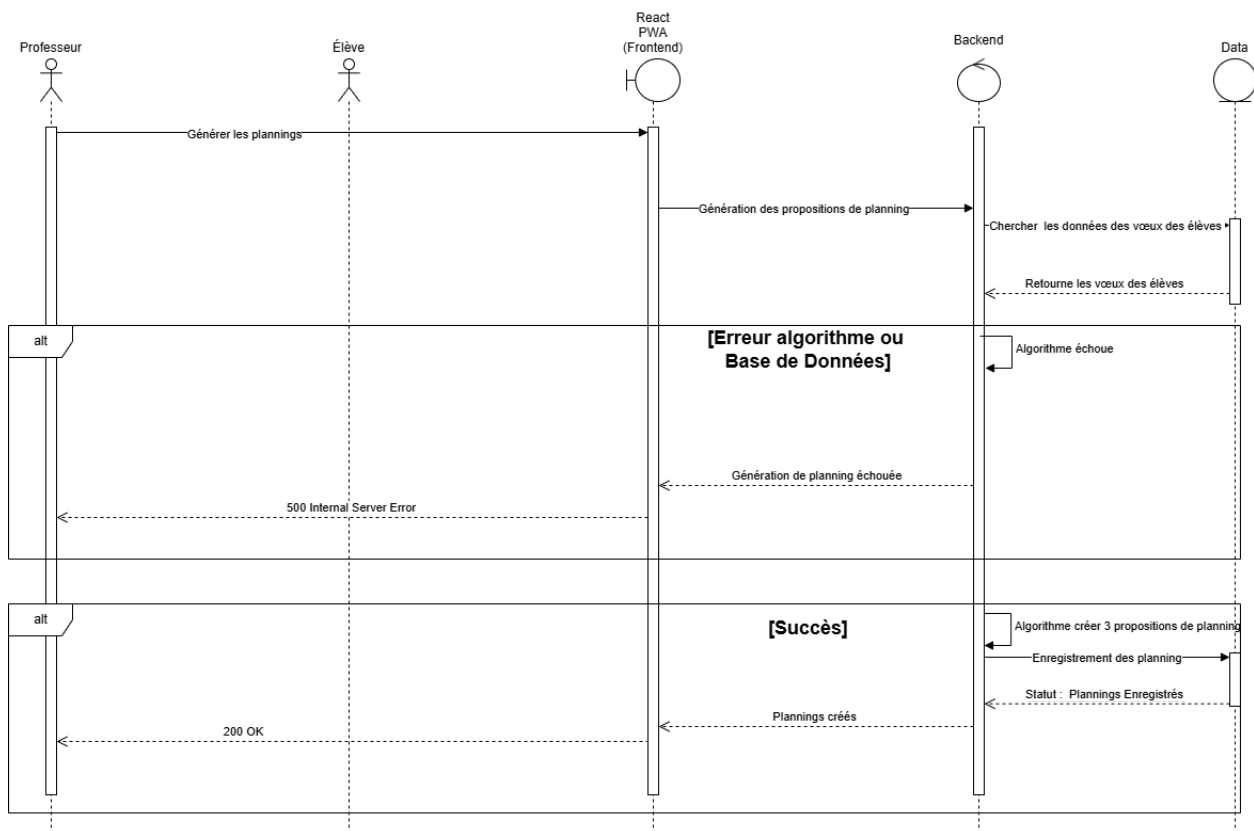


Diagramme de séquence : Validation par le professeur :

Diagramme de séquence : Validation du planning par le Professeur

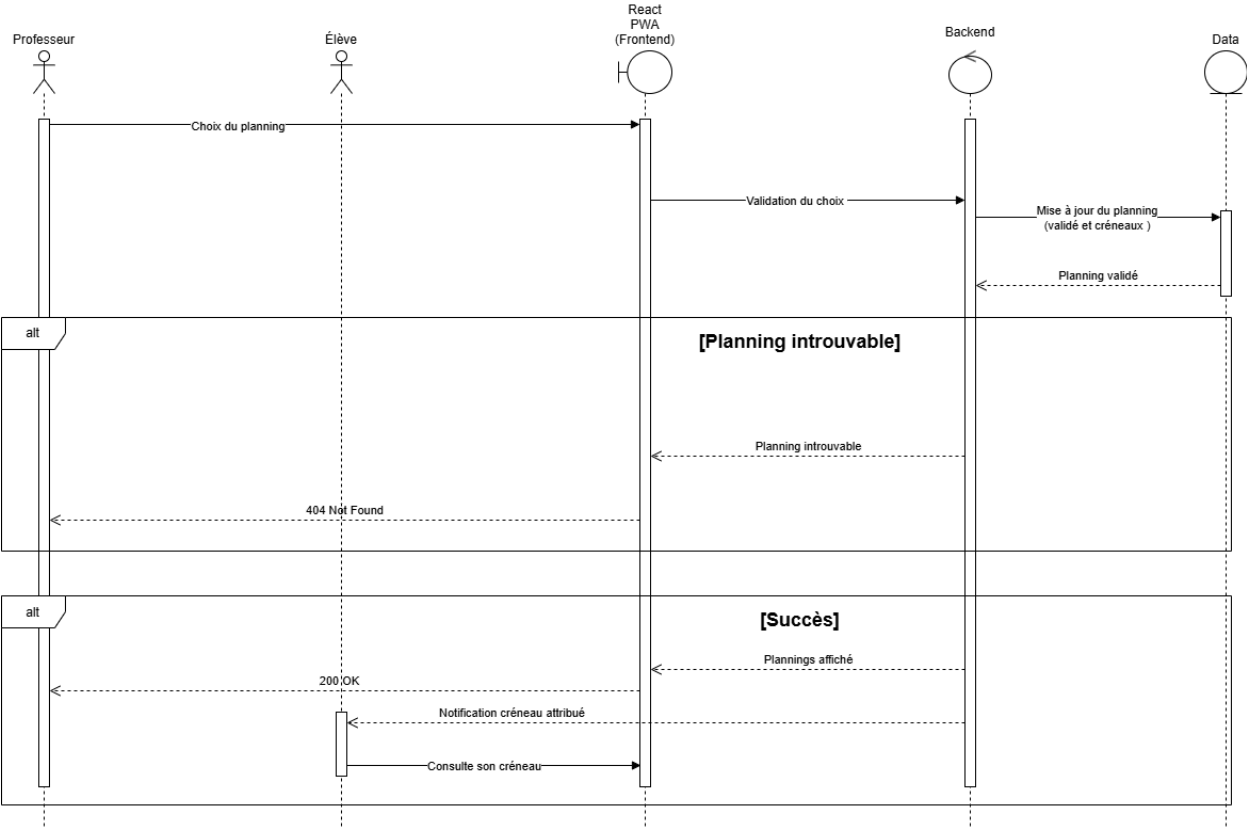


Diagramme de séquence : Suivi pédagogique et notification FCM (Firebase Cloud Messaging) :

Diagramme de séquence : Rédaction et envoi des objectifs:

Diagramme 3 : Suivi pédagogique et notification FCM :

Ce diagramme illustre le flux post-cours entre le professeur et l'élève. Le professeur clique sur un créneau depuis FullCalendar, rédige ses objectifs et conseils pédagogiques, et les sauvegarde en base de données.

- Les erreurs 400 (contenu invalide), 404 (créneau introuvable) et 500 (erreur base de données) sont gérées.
- En cas de succès, le backend envoie une notification push via Firebase Cloud Messaging. Si le token FCM de l'élève est expiré ou invalide, l'envoi échoue mais le flux n'est pas bloqué car l'objectif est déjà sauvegardé.
- L'élève consulte ensuite ses objectifs depuis son dashboard, avec gestion des cas 404 (aucun objectif trouvé) et 500 (erreur base de données).

Diagramme de séquence : Validation du planning par le Professeur

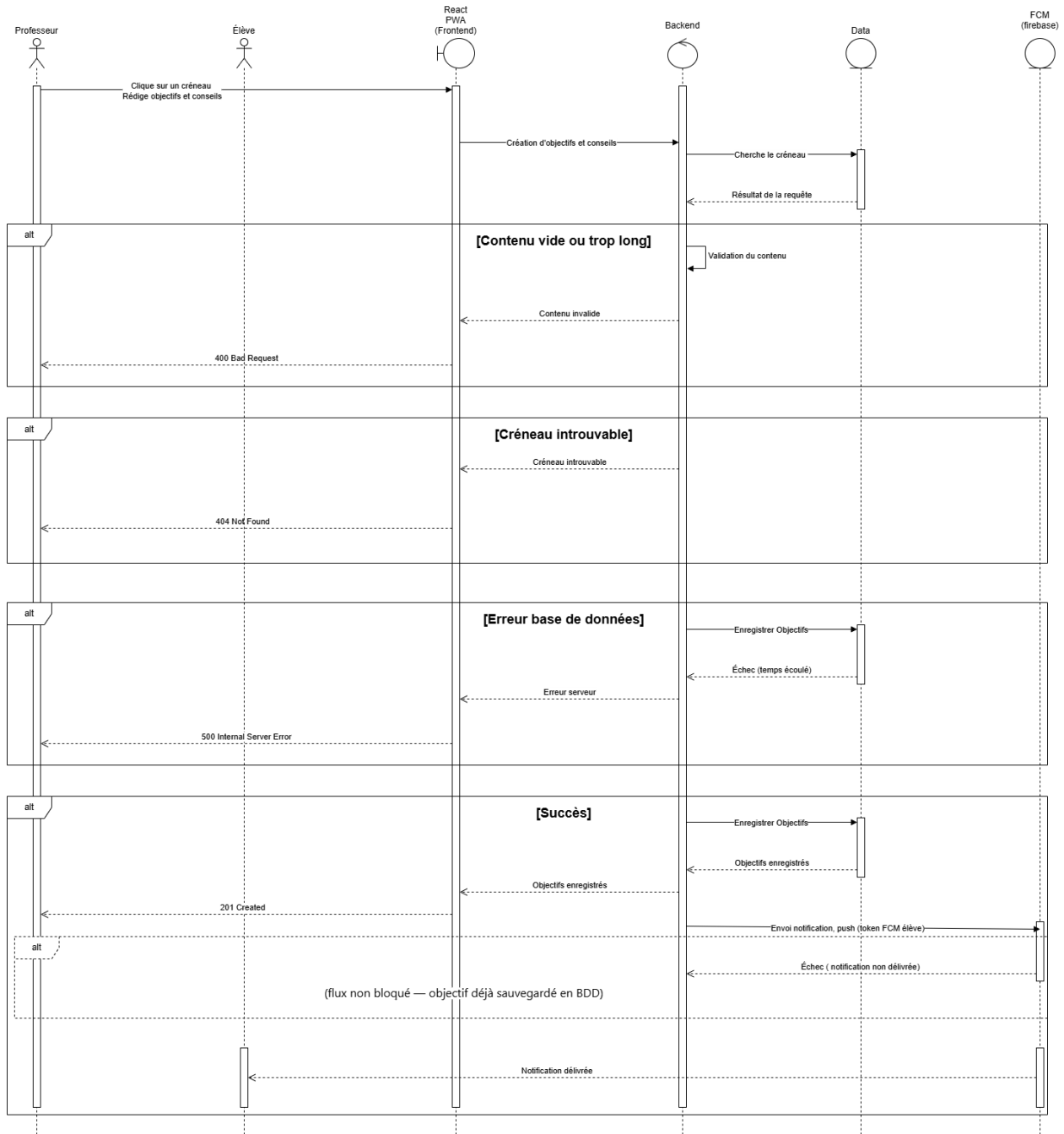
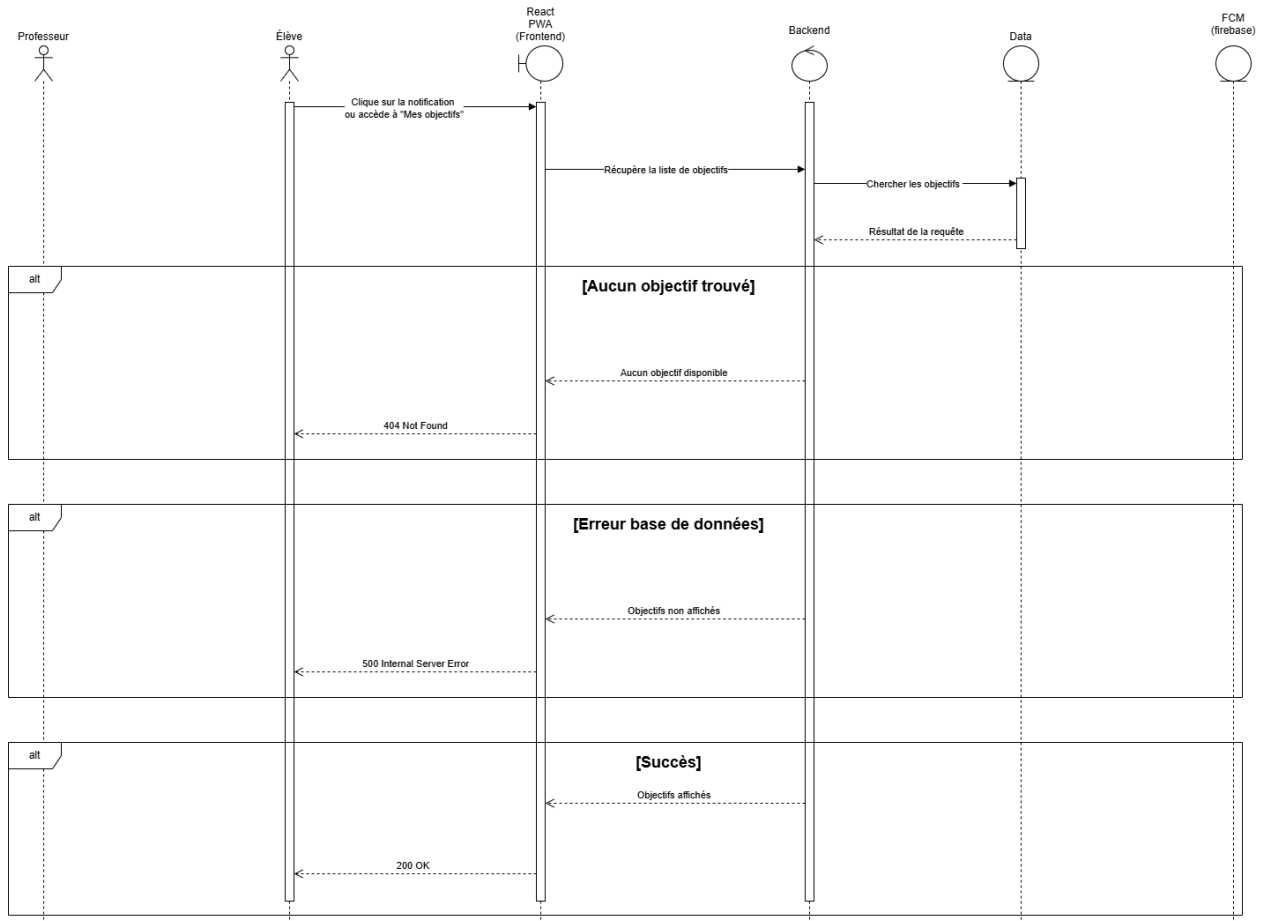


Diagramme de séquence : Consultations des objectifs:

Diagramme de séquence : Consultation des objectifs



VI.API externes et internes

Routes internes :

Les routes internes sont les routes exposées par le backend Flask et consommées par le frontend React PWA. Ils constituent l'API REST de PlanifPro et couvrent l'ensemble des fonctionnalités de l'application : authentification, gestion des classes, collecte des vœux, génération du planning, suivi pédagogique, événements et notifications. Toutes les routes protégées nécessitent un JWT valide transmis dans l'en-tête de la requête.

Auth endpoints (/auth/*)

Path	Met hod	Rôle	Input	Output	Usage
/auth/register	POST	Les deux	{prenom, nom, email, mot_de_passe, role}	201 {jwt_token, role}	Créer un compte utilisateur
/auth/login	POST	Les deux	{email, mot_de_passe}	200 {jwt_token, role}	Connecter un utilisateur

Profil endpoints (/profil/*)

Path	Met hod	Rôle	Input	Output	Usage
/profil	GET	Les deux	JWT header	200 {profil}	Récupérer son profil (nom, prénom, email)
/profil	PUT	Les deux	{prenom, nom, email}	200 {profil}	Modifier son profil
/profil	DELETE	Les deux	JWT header	200 {}	Supprimer son compte

Paramètres endpoints (/parametres/*)

Path	Met hod	Rôle	Input	Output	Usage
/parametres	GET	Les deux	JWT header	200 {parametres}	Récupérer ses paramètres (langue, notifications...)
/parametres	PUT	Les deux	{langue, notifications_push}	200 {parametres}	Modifier ses paramètres

Aide endpoints (/aide/*)

Path	Met hod	Rôle	Input	Output	Usage
/aide	GET	Les deux	JWT header	200 [{articles}]	Récupérer les articles d'aide / FAQ

Classes endpoints (/classes/*)

Path	Met hod	Rôle	Input	Output	Usage
/classes	GET	Professeur	JWT header	200 [{classes}]	Lister les classes du professeur
/classes	POST	Professeur	{nom, date_debut, date_fin, jours_horaires, nb_voeux_requis, nb_jours_min}	201 {classe_id, code_unique}	Créer une nouvelle classe
/classes/:id	GET	Les deux	JWT header	200 {classe}	Récupérer une classe par ID
/classes/:id	PUT	Professeur	{nom, date_debut, date_fin, jours_horaires}	200 {classe}	Modifier une classe existante
/classes/:id	DELETE	Professeur	JWT header	200 {}	Supprimer une classe
/classes/:id/eleves	GET	Professeur	JWT header	200 [{eleves}]	Lister les élèves d'une classe
/classes/:id/eleves	POST	Professeur	{eleve_id, duree_minutes}	201 {}	Ajouter un élève à une classe
/classes/rejoindre	POST	Élève	{code_unique}	201 {classe_id}	Élève rejoint une classe via code (interface classe)
/classes/:id/collecte	POST	Professeur	JWT header	200 {}	Lancer la collecte des vœux
/classes/:id/inviter	POST	Professeur	{email}	200 {}	Inviter un élève par email depuis l'interface classe

Élèves endpoints (/eleves/*)

Path	Met hod	Rôle	Input	Output	Usage
------	---------	------	-------	--------	-------

<code>/eleves</code>	GET	Professeur	JWT header	200 [{eleves}]	Lister tous ses élèves (toutes classes)
<code>/eleves/:id</code>	GET	Professeur	JWT header	200 {eleve}	Récupérer le profil d'un élève
<code>/eleves/:id</code>	PUT	Professeur	{duree_minutes}	200 {eleve}	Modifier la durée de cours d'un élève
<code>/eleves/:id</code>	DELETE	Professeur	JWT header	200 {}	Retirer un élève de toutes ses classes
<code>/eleves/inviter</code>	POST	Professeur	{email}	200 {}	Inviter un élève depuis le dashboard (SendGrid)

Professeurs endpoints (/professeurs/*)

Path	Method	Rôle	Input	Output	Usage
<code>/professeurs</code>	GET	Élève	JWT header	200 [{professeurs}]	Lister tous ses professeurs
<code>/professeurs/:id</code>	GET	Élève	JWT header	200 {professeur}	Récupérer le profil d'un professeur
<code>/professeurs/:id</code>	DELETE	Élève	JWT header	200 {}	Se désinscrire d'un professeur
<code>/professeurs/rejoindre</code>	POST	Élève	{code_unique}	201 {classe_id}	Rejoindre un professeur depuis le dashboard

Vœux endpoints (/voeux/*)

Path	Method	Rôle	Input	Output	Usage
<code>/voeux</code>	GET	Les deux	JWT header + ?classe_id	200 [{voeux}]	Lister les vœux d'une classe
<code>/voeux</code>	POST	Élève	{classe_id, creneaux_souhaitees}	201 {voeux_id}	Soumettre ses vœux d'horaires
<code>/voeux/:id</code>	GET	Les deux	JWT header	200 {voeux}	Récupérer un vœu par ID
<code>/voeux/:id</code>	PUT	Élève	{creneaux_souhaitees}	200 {voeux}	Modifier ses vœux (avant génération)
<code>/voeux/statut/:classe_id</code>	GET	Professeur	JWT header	200 [{statuts}]	Récupérer statut soumission par élève
<code>/voeux/relancer</code>	POST	Professeur	{classe_id, eleve_ids[]}	200 {}	Relancer les élèves en attente (FCM)

Planning endpoints (/planning/*)

Path	Met hod	Rôle	Input	Output	Usage
/planning/global	GET	Les deux	JWT header	200 [{creneaux}]	Récupérer le planning global (FullCalendar)
/planning/generer	POST	Professeur	{classe_id}	200 [{3 propositions}]	Générer 3 propositions de planning
/planning/valider	PUT	Professeur	{planning_id}	200 {planning}	Valider une proposition de planning
/planning/:id	GET	Les deux	JWT header	200 {planning}	Récupérer un planning par ID
/planning/:id/creneaux	GET	Les deux	JWT header	200 [{creneaux}]	Lister les créneaux d'un planning
/planning/:id/confirmation	PUT	Professeur	{confirmation_requise: bool}	200 {}	Activer/désactiver confirmation élève (US-15)

Créneaux endpoints (/creneaux/*)

Path	Met hod	Rôle	Input	Output	Usage
/creneaux	GET	Les deux	JWT header	200 [{creneaux}]	Lister tous les créneaux
/creneaux	POST	Professeur	{planning_id, eleve_id, jour, heure_debut, heure_fin, duree_minutes}	201 {creneau}	Créer un créneau manuellement
/creneaux/:id	GET	Les deux	JWT header	200 {creneau}	Récupérer un créneau par ID
/creneaux/:id	PUT	Professeur	{jour, heure_debut, heure_fin, periode}	200 {creneau}	Modifier un créneau (tout / un jour / plusieurs jours)
/creneaux/:id	DELETE	Professeur	{periode}	200 {}	Supprimer un créneau (tout / un jour / plusieurs jours)
/creneaux/echanger	PUT	Professeur	{creneau_id_1, creneau_id_2, periode}	200 {}	Échanger deux créneaux (tout / un jour / plusieurs jours)
/creneaux/:id/confirmer	PUT	Élève	JWT header	200 {}	Élève confirme son créneau attribué
/creneaux/:id/deplacer	PUT	Professeur	{nouveau_jour, nouvelle_heure_debut, nouvelle_heure_fin, periode}	200 {creneau}	Déplacer un créneau via drag and drop (FullCalendar)

Créneaux personnels endpoints (/creneaux/perso/*)

Path	Met hod	Rôle	Input	Output	Usage
/creneaux/perso	GET	Les deux	JWT header	200 [{creneaux}]	Lister ses créneaux personnels
/creneaux/perso	POST	Les deux	{jour, heure_debut, heure_fin, titre}	201 {creneau}	Ajouter un créneau personnel
/creneaux/perso/:id	PUT	Les deux	{jour, heure_debut, heure_fin, titre, periode}	200 {creneau}	Modifier un créneau personnel (tout / un jour / plusieurs jours)
/creneaux/perso/:id	DELETE	Les deux	{periode}	200 {}	Supprimer un créneau personnel (tout / un jour / plusieurs jours)
/creneaux/perso/:id/deplacer	PUT	Les deux	{nouveau_jour, nouvelle_heure_debut, nouvelle_heure_fin, periode}	200 {creneau}	Déplacer un créneau personnel via drag and drop
/creneaux/perso/import	POST	Les deux	{google_access_token}	200 [{creneaux}]	Importer son planning depuis Google Calendar

Objectifs endpoints (/objectifs/*)

Path	Met hod	Rôle	Input	Output	Usage
/objectifs	GET	Les deux	JWT header + ?eleve_id	200 [{objectifs}]	Lister les objectifs d'un élève
/objectifs	POST	Professeur	{creneau_id, contenu, conseils}	201 {objectif_id}	Rédiger un objectif après le cours
/objectifs/:id	GET	Les deux	JWT header	200 {objectif}	Récupérer un objectif par ID
/objectifs/:id	PUT	Professeur	{contenu, conseils}	200 {objectif}	Modifier un objectif existant
/objectifs/:id	DELETE	Professeur	JWT header	200 {}	Supprimer un objectif

Événements endpoints (/evenements/*)

Path	Met hod	Rôle	Input	Output	Usage
/evenements	GET	Les deux	JWT header	200 [{evenements}]	Lister les événements

<code>/evenements</code>	POST	Professeur	{titre, description, date_heure, destinataires}	201 {evenement_id}	Créer un événement (FCM notif.)
<code>/evenements/:id</code>	GET	Les deux	JWT header	200 {evenement}	Récupérer un événement par ID
<code>/evenements/:id</code>	PUT	Professeur	{titre, description, date_heure, destinataires}	200 {evenement}	Modifier un événement
<code>/evenements/:id</code>	DELETE	Professeur	JWT header	200 {}	Supprimer un événement

Notifications endpoints (/notifications/*)

Path	Method	Rôle	Input	Output	Usage
<code>/notifications</code>	GET	Les deux	JWT header	200 [{notifications}]	Lister les notifications de l'utilisateur
<code>/notifications/:id</code>	PUT	Les deux	{lu: true}	200 {}	Marquer une notification comme lue
<code>/notifications/lire</code>	PUT	Les deux	JWT header	200 {}	Marquer toutes les notifications comme lues
<code>/notifications/token</code>	POST	Les deux	{token_fcm}	200 {}	Enregistrer le token FCM de l'appareil

Google Calendar endpoints (/calendar/*)

Path	Method	Rôle	Input	Output	Usage
<code>/calendar/auth</code>	GET	Les deux	JWT header	200 {auth_url}	Obtenir l'URL d'autorisation OAuth 2.0
<code>/calendar/callback</code>	GET	Les deux	?code=oauth_code	200 {}	Callback OAuth — échange le code
<code>/calendar/export</code>	POST	Les deux	{creneaux_ids[]}	200 {event_ids[]}	Exporter les créneaux vers Google Calendar

Routes externes :

Les Routes externes sont les routes des services tiers appelées exclusivement depuis le backend Flask, jamais directement depuis le frontend. Ils permettent à PlanifPro de s'appuyer sur trois services spécialisés : Google Calendar API pour l'export des créneaux vers l'agenda personnel des utilisateurs via OAuth 2.0, SendGrid pour l'envoi des emails d'invitation et de relance, et Firebase Cloud Messaging pour l'ensemble des notifications push de l'application.

Google Calendar API — endpoints externes

Path	Method	Input	Output	Usage
<code>accounts.google.com/o/oauth2/auth</code>	GET	<code>?client_id,redirect_uri, scope</code>	302 → page autorisation Google	Rediriger l'utilisateur vers la page d'autorisation OAuth 2.0
<code>oauth2.googleapis.com/token</code>	POST	<code>{code, client_id, client_secret, redirect_uri}</code>	200 <code>{access_token, refresh_token}</code>	Échanger le code OAuth contre un access token
<code>www.googleapis.com/calendar/v3/calendars/primary/events</code>	POST	<code>{summary, start, end, description}</code>	200 <code>{event_id}</code>	Créer un événement dans l'agenda Google de l'utilisateur

SendGrid API — endpoints externes

Path	Method	Input	Output	Usage
<code>api.sendgrid.com/v3/mail/send</code>	POST	<code>{to, from, subject, content, template_id}</code>	202 {}	Envoyer un email d'invitation de classe ou de relance vœux

Firebase Cloud Messaging — endpoints externes

Path	Method	Input	Output	Usage
<code>fcm.googleapis.com/v1/projects/:project_id/messages:send</code>	POST	<code>{message: {token, notification: {title, body}}}</code>	200 {name}	Envoyer une notification push à un appareil via son token FCM

VII. Planification des stratégies de SCM et QA

SCM : Source Code Management :

La stratégie SCM de PlanifPro repose sur le workflow Gitflow, adapté au développement en solo. Elle définit l'organisation des branches, la convention des commits, le processus des pull requests et la gestion des issues GitHub.

1. Stratégie Git : Gitflow

PlanifPro adopte le workflow Gitflow qui organise le dépôt autour de deux branches permanentes et de branches temporaires créées à la demande.

Branches permanentes :

- **main** : contient uniquement le code de production stable et déployé sur Vercel (frontend) et Render (backend). Chaque merge sur main correspond à une mise en production.
- **develop** : branche d'intégration centrale. Toutes les branches feature fusionnent ici une fois terminées. Représente l'état le plus avancé du code en cours de développement.

Branches temporaires :

- **feature/*** : une branche par fonctionnalité, créée depuis develop et fusionnée dans develop après vérification.
- **release/*** : créée depuis develop quand une version est prête à être déployée. Permet les derniers ajustements avant fusion dans main.
- **hotfix/*** : créée depuis main pour corriger un bug critique en production. Fusionnée dans main ET dans develop.

Branch	Créée depuis	Fusionnée dans	Rôle
main	—	—	Code de production stable
develop	main	main (via release)	Intégration des features
feature/*	develop	develop	Développement d'une fonctionnalité
release/*	develop	main + develop	Préparation d'une version
hotfix/*	main	main + develop	Correction urgente en production

2. Nommage des branches

Convention de nommage : type/description-en-kebab-case

Type	Format	Exemples
------	--------	----------

Feature	feature/nom-fonctionnalite	feature/auth, feature/voeux, feature/planning-algo
Release	release/vX.X.X	release/v1.0.0, release/v1.1.0
Hotfix	hotfix/description-bug	hotfix/fix-jwt-expiry, hotfix/fix-fcm-token

Liste des branches feature de PlanifPro :

- **feature/auth** : inscription, connexion, déconnexion, JWT
- **feature/profil** : profil utilisateur, paramètres, aide
- **feature/classes** : création et gestion des classes
- **feature/eleves** : gestion des élèves, invitations
- **feature/professeurs** : gestion des professeurs côté élève
- **feature/voeux** : collecte et soumission des vœux
- **feature/planning-algo** : algorithme de génération des 3 propositions
- **feature/planning-validation** : validation, confirmation élève
- **feature/creneaux** : gestion des créneaux attribués, drag and drop
- **feature/creneaux-perso** : créneaux personnels, import Google Calendar
- **feature/objectifs** : suivi pédagogique
- **feature/evenements** : création et diffusion d'événements
- **feature/notifications** : FCM, centre de notifications
- **feature/google-calendar** : export OAuth 2.0

3. Convention des commits

PlanifPro suit la convention Conventional Commits. Chaque message de commit respecte le format suivant :

type(scope) : description courte en minuscules

Type	Usage	Exemple PlanifPro
feat	Nouvelle fonctionnalité	feat(auth): add JWT login endpoint
fix	Correction de bug	fix(planning): fix algo crash on empty voeux
refactor	Refactoring sans changement de comportement	refactor(creneaux): simplify drag drop handler
test	Ajout ou modification de tests	test(voeux): add unit tests for soumission
docs	Documentation	docs(api): update routes auth documentation
style	Formatage, espaces (pas de logique)	style(dashboard): fix tailwind class order
chore	Maintenance, dépendances	chore: update flask-jwt-extended to 4.6
hotfix	Correction urgente en production	hotfix(fcm): fix expired token not handled

Règles additionnelles :

- La description est en anglais, en minuscules, sans point final
- Le scope correspond au module concerné : auth, classes, voeux, planning, creneaux, objectifs, evenements, notifications, profil
- Maximum 72 caractères pour la ligne de titre
- Un commit = une seule modification logique

4. Pull Requests

PlanifPro étant développé en solo, les pull requests sont ouvertes et mergées par le développeur lui-même. Elles servent principalement à déclencher la pipeline CI/CD automatiquement avant chaque merge, et à conserver un historique clair et structuré des fonctionnalités ajoutées.

Nommage des PR :

`[type] Description courte – ex: [feat] Ajout de la génération du planning`

Checklist avant merge d'une PR :

- Le code compile et tourne localement sans erreur
- Les tests unitaires passent
- Les routes API ont été testées avec curl
- La pipeline CI/CD GitHub Actions passe avec succès
- La branche est à jour avec develop
- Les migrations de base de données sont incluses si nécessaire
- La PR est liée à une issue GitHub (ex: Closes #12)

Règles de merge :

- Le merge se fait uniquement si la pipeline CI/CD est verte
- Le merge se fait en Squash and merge pour garder un historique propre sur develop
- La branche feature est supprimée après le merge

5. Git Issues

Les issues GitHub servent à tracker les fonctionnalités à développer, les bugs et les améliorations. Chaque branche feature correspond à une issue.

Labels utilisés :

Label	Couleur	Usage
feature	Vert	Nouvelle fonctionnalité à développer

bug	Rouge	Comportement incorrect à corriger
enhancement	Bleu	Amélioration d'une fonctionnalité existante
documentation	Jaune	Mise à jour de la documentation
hotfix	Orange	Correction urgente en production
wontfix	Gris	Problème connu mais non traité dans cette version

Template d'issue :

Titre : [type] Description courte

Description : Contexte et objectif de l'issue

Critères d'acceptation : liste des conditions pour considérer l'issue terminée

Branche associée : feature/nom-fonctionnalité

Lié à : #numéro-issue-parente (si applicable)

Lien commits / issues :

- Un commit peut référencer une issue avec #numéro : feat(auth): add register endpoint #3
- Un commit peut fermer automatiquement une issue avec Closes #numéro ou Fixes #numéro
- Une PR doit toujours mentionner Closes #numéro dans sa description pour fermer l'issue au merge

QA : Quality Assurance :

La stratégie QA de PlanifPro repose sur trois niveaux de tests adaptés au contexte d'un projet développé en solo : tests unitaires automatisés, tests des routes API avec curl, et tests end-to-end manuels.

1. Outils de tests

Couche	Outil	Usage
Backend (Flask)	pytest + pytest-flask	Tests unitaires Python automatisés
Frontend (React)	Jest + React Testing Library	Tests unitaires composants React automatisés
Routes API	curl	Tests manuels des routes HTTP depuis le terminal
End-to-end	Tests manuels navigateur	Validation des parcours utilisateur complets
Couverture	pytest-cov + Istanbul	Rapport de couverture de code
CI/CD	GitHub Actions	Exécution automatique des tests unitaires à chaque PR
E2E futur (V2)	Cypress	Automatisation des tests end-to-end prévue en V2

2. Tests unitaires

Les tests unitaires vérifient le comportement isolé de chaque fonction ou composant, sans dépendances externes (base de données, APIs tierces). Ils sont exécutés automatiquement par GitHub Actions à chaque pull request.

Backend : pytest :

- **Routes Auth** : inscription (email valide/valide, email existant, mdp trop court), connexion (succès, mauvais mdp, utilisateur introuvable)
- **Algorithme de planning** : génération des 3 propositions avec vœux valides, gestion des cas limites (vœux insuffisants, créneaux incompatibles)
- **Validation des vœux** : respect des contraintes (nombre minimum, jours différents)
- **Utilitaires** : hashage bcrypt, génération JWT, génération code unique de classe

Frontend : Jest + React Testing Library :

- **Composants formulaires** : PageConnexion, PageInscription (validation des champs, affichage des erreurs)
- **Composants dashboard** : affichage conditionnel selon le rôle (Professeur / Élève)
- **Composant formulaire vœux** : sélection des créneaux, comptage, validation des contraintes
- **Fonctions utilitaires** : formatage des dates, calcul des durées de créneaux

3. Tests des routes API : curl

Les routes API sont testées manuellement avec curl pendant le développement. curl est un outil en ligne de commande qui permet d'envoyer des requêtes HTTP directement depuis le terminal et de vérifier les réponses du backend Flask.

Exemples de commandes curl pour PlanifPro :

Inscription :

```
curl -X POST http://localhost:5000/auth/register \
  -H "Content-Type: application/json" \
  -d '{"prenom":"Pauline","nom":"M","email":"p@test.com","mot_de_passe":"mdp123","role":"professeur"}'
```

Connexion et récupération du token JWT :

```
curl -X POST http://localhost:5000/auth/login \
  -H "Content-Type: application/json" \
  -d '{"email":"p@test.com","mot_de_passe":"mdp123"}'
```

Route protégée avec JWT :

```
curl -X GET http://localhost:5000/classes \
  -H "Authorization: Bearer <jwt_token>"
```

Soumission de vœux :

```
curl -X POST http://localhost:5000/voeux \
  -H "Content-Type: application/json" \
  -H "Authorization: Bearer <jwt_token>" \
  -d '{"classe_id":"uuid-classe","creneaux_souhaites":[...]}'
```

Cas d'erreur à vérifier pour chaque route :

- Réponse 400 : données manquantes ou invalides
- Réponse 401 : token JWT absent ou expiré
- Réponse 404 : ressource introuvable
- Réponse 409 : conflit (email existant, planning déjà généré...)
- Réponse 500 : erreur serveur

4. Tests end-to-end : Manuels

Les tests end-to-end sont effectués manuellement dans le navigateur par le développeur avant chaque merge dans main. Ils valident les parcours complets d'utilisation de l'application.

Parcours Professeur :

- Inscription → connexion → création d'une classe → invitation d'un élève
- Lancement de la collecte des vœux → vérification de la notification élève
- Génération des 3 propositions → validation d'un planning → vérification des créneaux attribués
- Rédaction d'un objectif → vérification de la notification élève
- Création d'un événement → vérification de l'affichage dans le calendrier élève
- Export vers Google Calendar

Parcours Élève :

- Inscription → connexion → rejoindre une classe via code unique
- Soumission des vœux → vérification du statut soumis côté professeur
- Confirmation du créneau attribué → vérification de l'affichage dans le calendrier
- Consultation des objectifs → vérification de l'historique chronologique
- Ajout d'un créneau personnel → drag and drop dans FullCalendar

Note : l'automatisation de ces tests avec Cypress est prévue en V2.

5. Intégration continue : GitHub Actions

À chaque pull request vers develop ou main, GitHub Actions exécute automatiquement la pipeline suivante :

- Linting → vérification du style de code (flake8 pour Python, ESLint pour React)
- Tests unitaires → pytest (backend) + Jest (frontend)
- Rapport de couverture → résultat affiché directement dans la PR
- Build → vérification que l'application compile sans erreur

Le merge dans develop ou main est bloqué si la pipeline échoue.

VIII. Justifications techniques

Vue d'ensemble de la stack technique

Couche	Technologie choisie	Alternatives écartées
Frontend	React PWA	React Native, Angular, Vue
Déploiement front	Vercel	Netlify, AWS Amplify
Calendrier	FullCalendar	Développement custom, DHTMLX
Style	Tailwind CSS	Bootstrap, CSS classique, MUI
Backend	Python / Flask	Django, Node.js / Express
Authentification	JWT	Sessions, OAuth2 seul
ORM	SQLAlchemy	Requêtes SQL brutes, Peewee
Base de données	PostgreSQL	MySQL, MongoDB, SQLite
Déploiement back	Render	AWS, Heroku, Railway
Emails	SendGrid	Mailgun, Nodemailer, SMTP
Notifications push	FCM (Firebase)	OneSignal, Pusher, Web Push API
Export calendrier	Google Calendar API	iCal, Outlook API

Frontend

1. React PWA vs React Native / Angular / Vue

React PWA a été retenu comme solution frontend pour PlanifPro pour les raisons suivantes :

- Un seul codebase pour web et mobile : la PWA est installable depuis le navigateur sur iOS et Android sans passer par l'App Store ou le Google Play Store, ce qui est idéal pour un MVP.
- React Native aurait nécessité un codebase séparé du web, doublant la charge de développement et de maintenance pour un projet solo.
- Angular est un framework plus lourd et plus verbeux, conçu pour de grandes équipes et des applications d'entreprise. Sa courbe d'apprentissage est plus élevée pour un gain limité sur ce projet.
- Vue est une alternative sérieuse mais l'écosystème React est plus large, avec davantage de bibliothèques compatibles (FullCalendar, Tailwind, etc.) et une communauté plus active.
- La PWA permet au Service Worker de gérer les notifications push FCM et la mise en cache des ressources, deux besoins critiques de PlanifPro.

2. Vercel vs Netlify / AWS Amplify

Vercel a été choisi pour le déploiement du frontend React PWA pour les raisons suivantes :

- Déploiement automatique depuis GitHub à chaque push sur main, aucune configuration manuelle requise.
- Optimisé nativement pour les applications React et Next.js, avec des performances de build supérieures à Netlify.
- Tier gratuit généreux et suffisant pour un MVP, sans carte bancaire requise.
- AWS Amplify est plus puissant mais beaucoup plus complexe à configurer et à maintenir pour un projet solo.

3. FullCalendar vs développement custom / DHTMLX

FullCalendar a été retenu comme composant de calendrier interactif pour les raisons suivantes :

- Bibliothèque open source mature et activement maintenue, avec une intégration React native via `@fullcalendar/react`.
- Supporte nativement les vues semaine et mois, le drag and drop des événements, et l'affichage des créneaux horaires, toutes des fonctionnalités critiques de PlanifPro.
- Développer un calendrier interactif from scratch représenterait plusieurs semaines de travail pour un résultat moins robuste.
- DHTMLX Scheduler est une alternative commerciale puissante mais payante, ce qui n'est pas adapté à un MVP.

4. Tailwind CSS vs Bootstrap / CSS classique / MUI

Tailwind CSS a été choisi pour le style de l'interface pour les raisons suivantes :

- Approche utility-first qui permet de construire des interfaces sur mesure rapidement, sans écrire de CSS personnalisé.
- Bootstrap impose une esthétique générique reconnaissable que Tailwind permet d'éviter, pour un rendu plus unique.
- MUI (Material UI) impose le design system de Google, ce qui entre en conflit avec la charte graphique bleue et dorée de PlanifPro.
- Tailwind génère uniquement le CSS utilisé, ce qui résulte en des fichiers de style très légers en production.

Backend

5. Python / Flask vs Django / Node.js Express

Flask a été retenu comme framework backend pour les raisons suivantes :

- Flask est un micro-framework minimaliste qui expose uniquement ce dont on a besoin. Il est idéal pour construire une API REST sans la complexité d'un framework full-stack.
- Django est beaucoup plus lourd et impose une structure rigide (ORM, admin, templates) dont PlanifPro n'a pas besoin. Il est conçu pour des applications web complètes, pas uniquement pour une API.
- Python est le langage le plus adapté pour implémenter l'algorithme de génération du planning, grâce à sa lisibilité et à ses bibliothèques de traitement de données.
- Node.js avec Express est une alternative viable mais Python offre un avantage significatif pour la logique algorithmique au cœur de PlanifPro.
- L'écosystème Python (SQLAlchemy, Flask-JWT-Extended, Firebase Admin SDK, SendGrid SDK) couvre parfaitement tous les besoins techniques du projet.

6. JWT vs Sessions / OAuth2 seul

JSON Web Tokens (JWT) ont été choisis pour l'authentification pour les raisons suivantes :

- JWT est stateless, le serveur n'a pas besoin de stocker les sessions en base de données, ce qui simplifie l'architecture et améliore les performances.
- Le token contient l'identifiant et le rôle de l'utilisateur, ce qui permet au backend de distinguer Professeur et Élève sans requête supplémentaire en base.
- Les sessions serveur nécessitent un stockage partagé (Redis, base de données) entre plusieurs instances, ce qui complique le déploiement sur Render.
- OAuth2 seul (Google, GitHub) aurait exclu les utilisateurs sans compte Google, ce qui n'est pas adapté à PlanifPro qui cible des professeurs de musique et des parents.
- JWT est la convention standard pour les APIs REST modernes et est nativement supportée par Flask-JWT-Extended.

7. SQLAlchemy vs requêtes SQL brutes /

SQLAlchemy a été choisi comme ORM pour les raisons suivantes :

- SQLAlchemy permet de manipuler la base de données en Python sans écrire de SQL brut, ce qui réduit les risques d'erreurs et d'injections SQL.
- Les modèles SQLAlchemy correspondent directement aux classes Python du diagramme UML, ce qui rend le code plus lisible et maintenable.

- Les migrations de schéma sont gérées par Flask-Migrate (basé sur Alembic), ce qui facilite les évolutions de la base de données.
- Les requêtes SQL brutes sont plus performantes dans certains cas mais beaucoup plus difficiles à maintenir sur le long terme.

Base de données

8. PostgreSQL vs MySQL / MongoDB / SQLite

PostgreSQL a été retenu comme base de données pour les raisons suivantes :

- Les données de PlanifPro sont fortement relationnelles : classes, élèves, vœux, créneaux, plannings sont liés par de nombreuses relations avec cardinalités. PostgreSQL est la base relationnelle la plus robuste pour ce type de modèle.
- PostgreSQL supporte nativement les UUID, les types JSON (pour les jours horaires et créneaux souhaités), et les contraintes d'unicité, toutes utilisées dans le schéma de PlanifPro.
- MySQL est une alternative sérieuse mais PostgreSQL est mieux supporté par Render et offre des fonctionnalités avancées (types natifs, performances sur les jointures complexes) supérieures.
- MongoDB (NoSQL) est inadapté car les données de PlanifPro nécessitent des transactions ACID et des jointures entre tables, deux points forts de PostgreSQL.
- SQLite est adapté au développement local mais ne convient pas à un déploiement en production multi-utilisateurs.

Déploiement

9. Render vs AWS / Heroku / Railway

Render a été choisi pour l'hébergement du backend Flask et de la base de données PostgreSQL pour les raisons suivantes :

- Render héberge le backend Python et la base de données PostgreSQL au même endroit, ce qui simplifie la configuration et réduit la latence entre les deux composants.
- Déploiement automatique depuis GitHub à chaque push, sans configuration complexe de pipelines CI/CD.
- Tier gratuit suffisant pour un MVP, avec des ressources dédiées et une bonne disponibilité.
- Heroku a supprimé son tier gratuit en 2022, ce qui le rend moins adapté à un projet MVP en phase de validation.

- AWS est beaucoup plus puissant mais sa complexité de configuration (EC2, RDS, VPC, IAM) est disproportionnée pour un projet solo en phase MVP.
- Railway est une alternative proche de Render mais avec une communauté plus restreinte et moins de documentation.

Services externes

10. SendGrid vs Mailgun / Nodemailer / SMTP

SendGrid a été choisi pour l'envoi d'emails pour les raisons suivantes :

- SDK Python officiel simple à intégrer dans Flask, avec une documentation claire et des exemples concrets.
- Tier gratuit de 100 emails par jour, suffisant pour les besoins du MVP (invitations de classe, relances de vœux).
- Taux de délivrabilité élevé, les emails SendGrid arrivent rarement en spam contrairement à un serveur SMTP classique.
- Mailgun est une alternative comparable mais son tier gratuit est limité à 5 000 emails sur 3 mois puis payant.
- Nodemailer est conçu pour Node.js et n'est pas adapté à un backend Python.
- Un serveur SMTP classique nécessite une configuration de domaine et des enregistrements DNS complexes non adaptés à un MVP.

11. FCM (Firebase Cloud Messaging) vs OneSignal / Pusher / Web Push API

Firebase Cloud Messaging a été choisi pour les notifications push pour les raisons suivantes :

- Service gratuit et fiable de Google, avec un SDK Firebase Admin Python officiel facile à intégrer dans Flask.
- FCM supporte tous les navigateurs modernes (Chrome, Firefox, Safari) et tous les appareils mobiles via le Service Worker de la PWA.
- OneSignal est une alternative puissante mais son SDK ajoute une dépendance externe côté frontend que FCM évite grâce au Service Worker natif.
- Pusher est davantage orienté temps réel (websockets) que notifications push, ce qui ne correspond pas au besoin de PlanifPro.
- La Web Push API standard sans FCM nécessite de gérer soi-même le serveur VAPID et la gestion des tokens, ce qui est plus complexe que d'utiliser FCM.

12. Google Calendar API vs iCal / Outlook API

Google Calendar API a été choisi pour l'export du planning pour les raisons suivantes :

- Google Calendar est l'agenda le plus utilisé en France, ce qui correspond au public cible de PlanifPro (professeurs de musique et parents).
- L'API Google Calendar est bien documentée, avec un SDK Python officiel et un flux OAuth 2.0 standard.
- iCal est un format universel mais il génère uniquement un fichier statique à importer manuellement, sans synchronisation automatique ni mise à jour en temps réel.
- L'API Outlook / Microsoft 365 est une alternative viable mais son intégration est plus complexe et son taux d'adoption est plus faible que Google Calendar dans le contexte de l'enseignement musical.

Conclusion

PlanifPro est une application de gestion de planning pédagogique conçue pour simplifier le quotidien des professeurs de musique et de leurs élèves.

Ce document technique a couvert l'ensemble des aspects de la conception du MVP : les user stories et mockups Figma, l'architecture système C4, le diagramme de classes UML et le schéma de base de données PostgreSQL, les composants frontend React, les diagrammes de séquence des flux critiques, la documentation complète des routes API REST, ainsi que les stratégies SCM et QA adaptées à un développement en solo.

Les technologies retenues : React PWA, Flask, PostgreSQL, Vercel, Render, SendGrid, FCM et Google Calendar API : ont été choisies pour leur adéquation au contexte d'un MVP, leur simplicité de mise en œuvre et leur coût maîtrisé.

En perspective, la V2 de PlanifPro pourrait intégrer un système de paiement en ligne via Stripe, une messagerie instantanée entre professeur et élève, une application mobile native React Native, ainsi que l'automatisation des tests end-to-end avec Cypress.